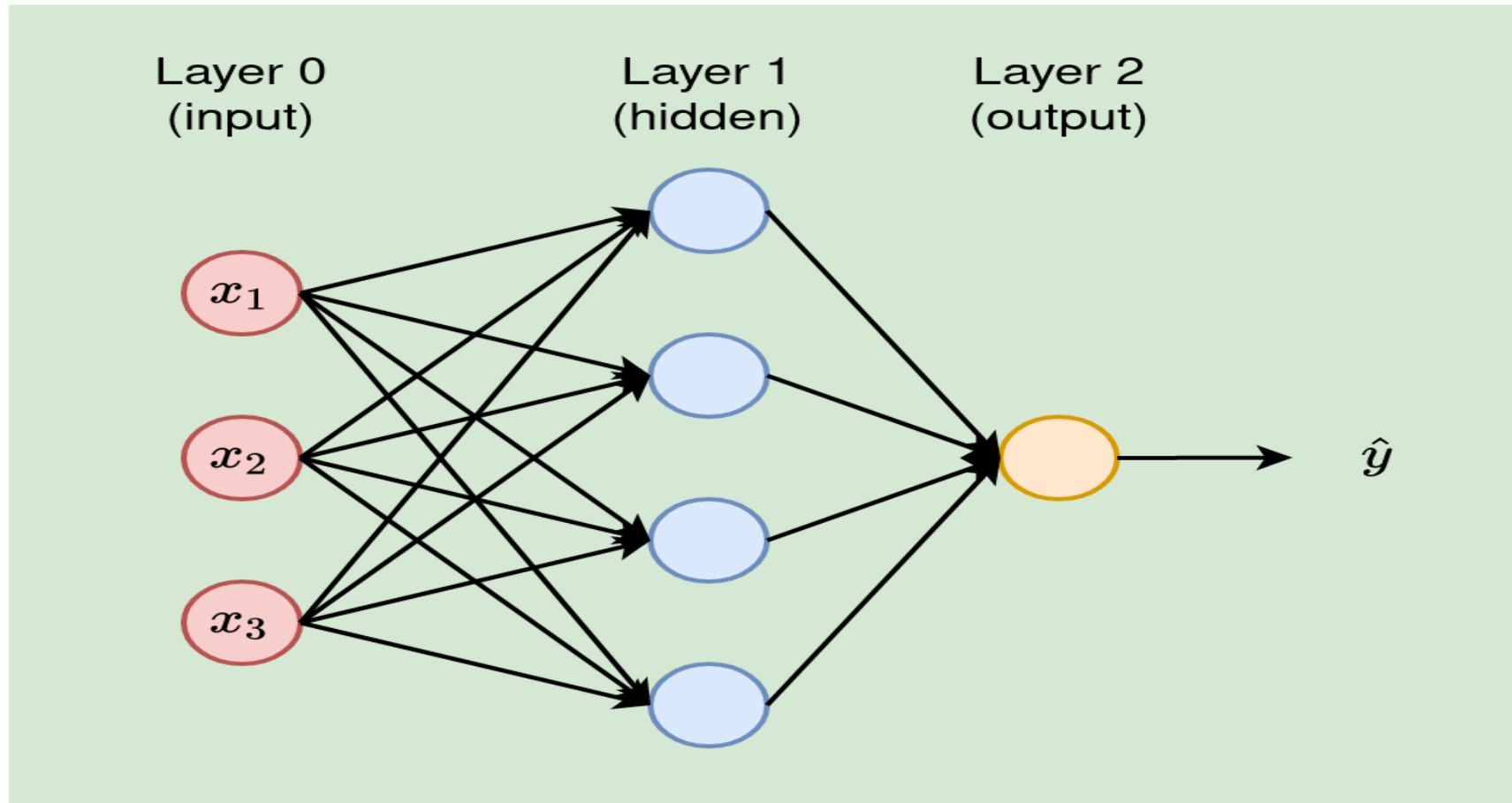


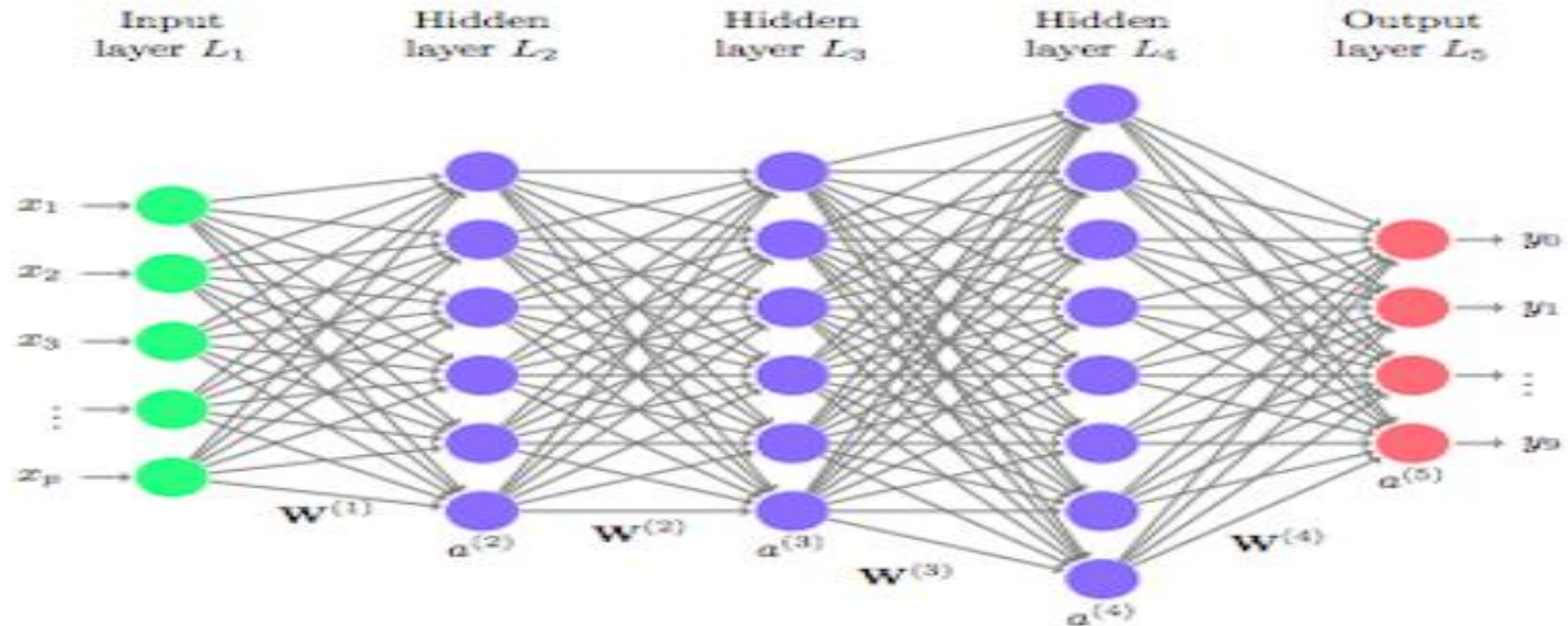
"THE WORLD IS
ONE **BIG DATA**
PROBLEM."

Training, Optimization & Regularization of DNN

Training of DNN



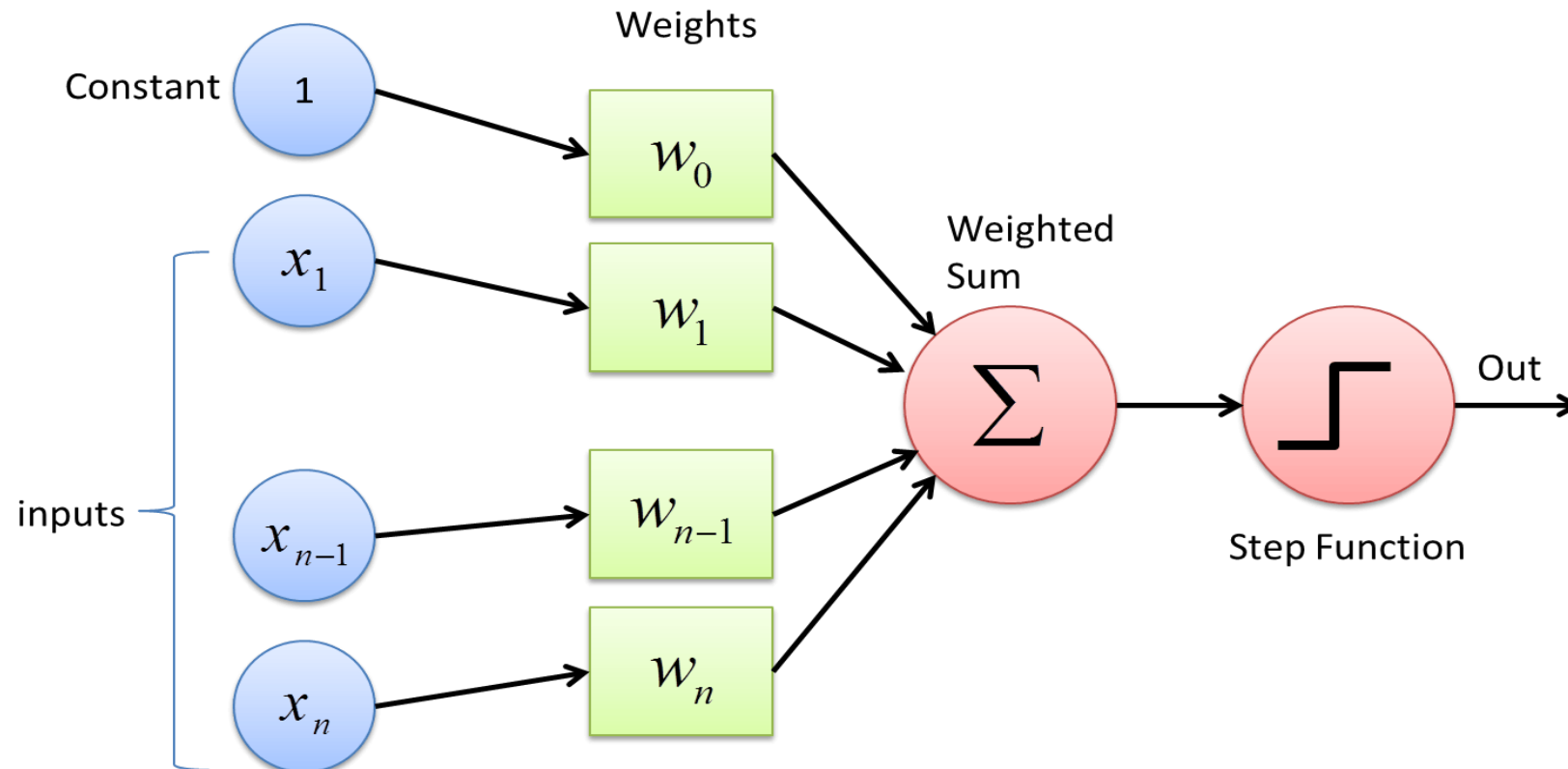
Training of DNN



Learning factors

- Initial weight
- Fixing up the desired output
- Non separable parameters
- Learning constant
- Momentum
- Activation Function

Training of DNN

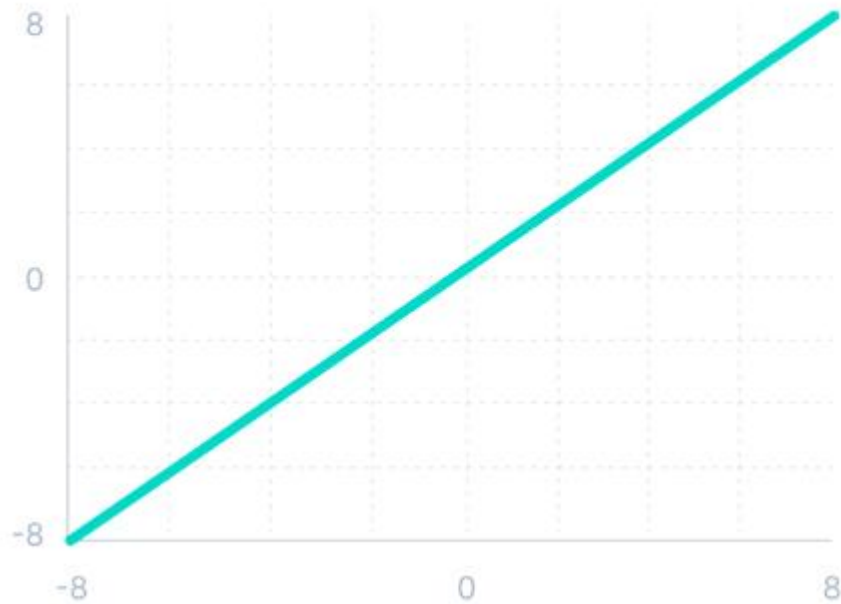


Activation Functions

- **Linear function**
- **Sigmoid / Logistic Activation Function** – Smooth Function
- **Tanh Function (Hyperbolic Tangent)** -- Smooth Function
- **Rectified Linear Unit (ReLU) Function** – Piecewise Linear Function, Rectifier
- **Leaky ReLU Function** – Piecewise Linear Function, Rectifier
- **Softmax Function** – Not activation Function but operation at o/p neurons

Linear function

Linear Activation Function



Mathematically it can be represented as: $f(x) = x$

"no activation," or "identity function"

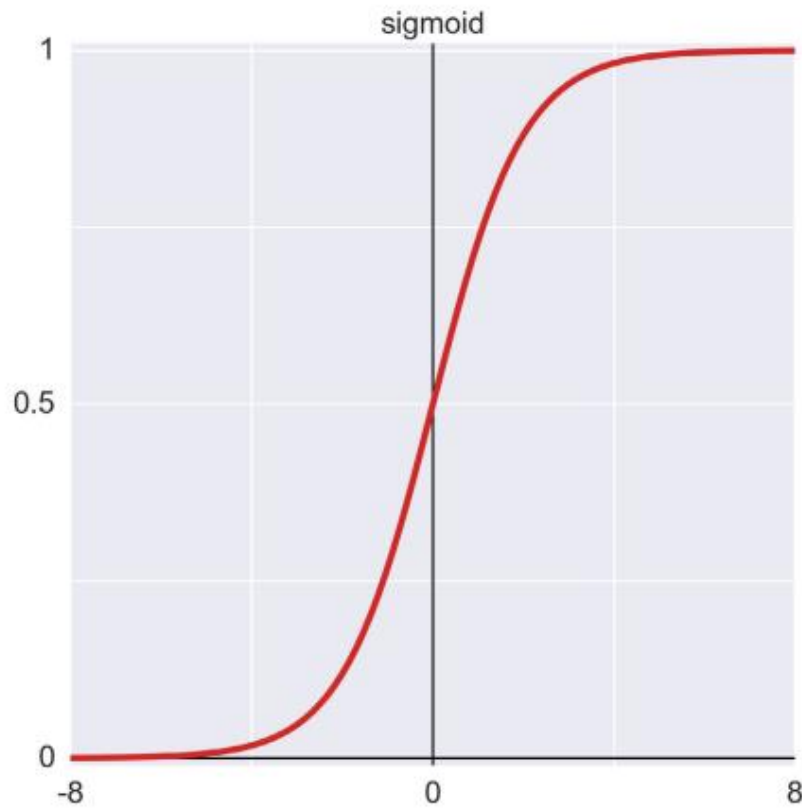
Cons of Linear function

- ❑ It's not possible to use backpropagation as the derivative of the function is a constant and has no relation to the input x .
- ❑ All layers of the neural network will collapse into one if a linear activation function is used. No matter the number of layers in the neural network, the last layer will still be a linear function of the first layer. So, essentially, a linear activation function turns the neural network into just one layer.

Non-Linear Activation Functions

- They allow backpropagation because now the derivative function would be related to the input, and it's possible to go back and understand which weights in the input neurons can provide a better prediction.
- They allow the stacking of multiple layers of neurons as the output would now be a non-linear combination of input passed through multiple layers. Any output can be represented as a functional computation in a neural network.

Logistic / Sigmoid function



Mathematically it can be represented as:

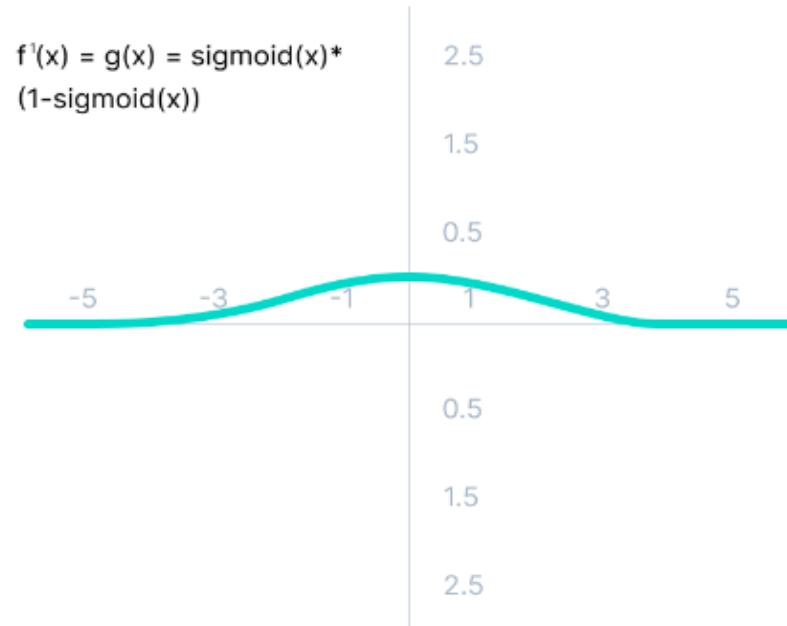
$$f(x) = \frac{1}{1 + e^{-x}}$$

Pros of Sigmoid function

- It is commonly used for models where we have **to predict the probability as an output**. Since probability of anything exists only between the range of 0 and 1, sigmoid is the right choice because of its range.
- The function is **differentiable and provides a smooth gradient**, i.e., preventing jumps in output values. This is represented by an S-shape of the sigmoid activation function.

Cons of Sigmoid function

The derivative of the function is
 $f'(x) = \text{sigmoid}(x) * (1 - \text{sigmoid}(x))$.

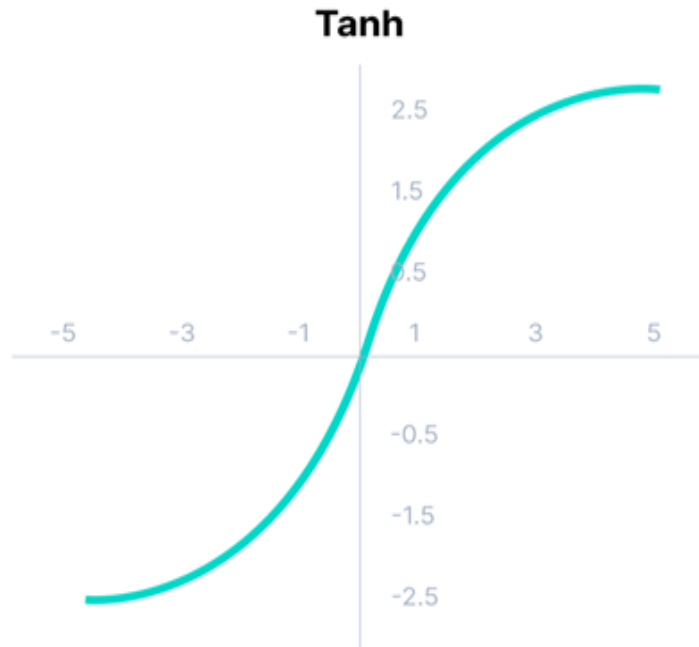


As the gradient value approaches zero, the network ceases to learn and suffers from the **Vanishing gradient problem**.

The output of the logistic function is **not symmetric around zero**. So the output of all the neurons will be of the same sign.

This makes the **training** of the neural network **more difficult and unstable**.

Tanh / Hyperbolic tangent function



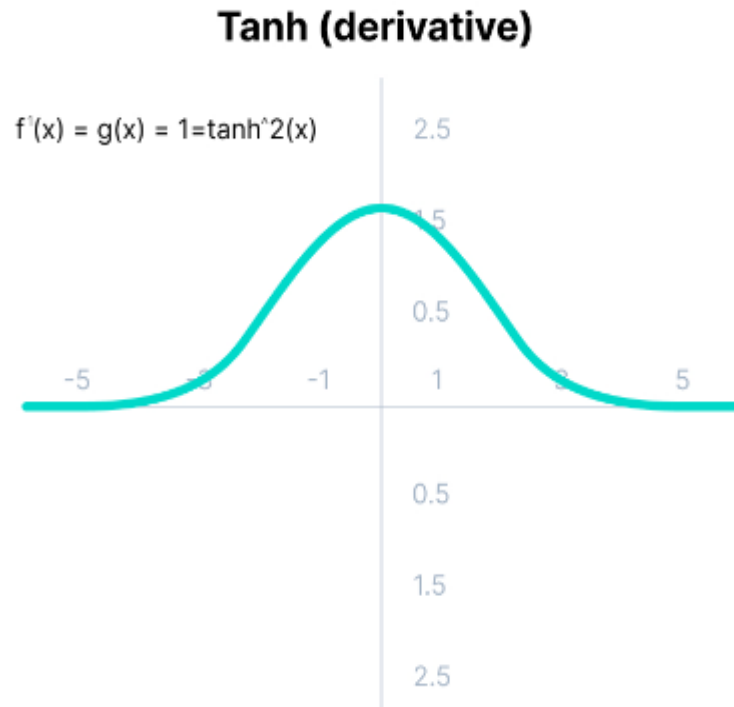
Mathematically it can be represented as:

$$f(x) = \frac{(e^x - e^{-x})}{(e^x + e^{-x})}$$

Advantages of Tanh function

- The output of the tanh activation function is **Zero centered**; hence we can easily map the output values as strongly negative, neutral, or strongly positive.
- Usually used in hidden layers of a neural network as its values lie between -1 to; therefore, the mean for the hidden layer comes out to be 0 or very close to it. It helps in **centering the data and makes learning for the next layer much easier.**

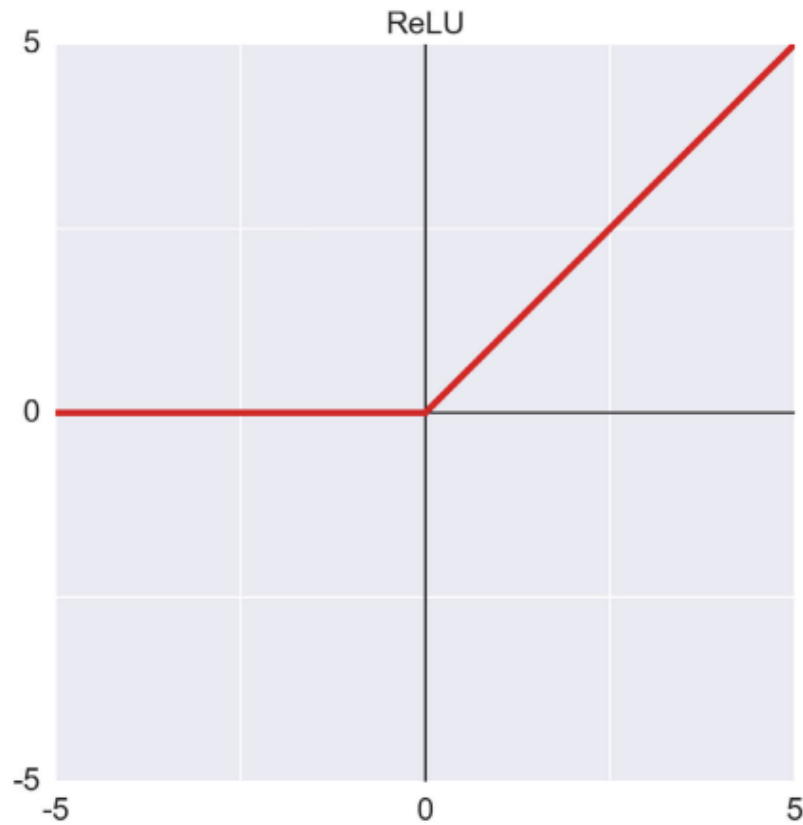
Have a look at the gradient of the tanh activation function



From the graph it is clear that it also faces the **problem of vanishing gradients** similar to the sigmoid activation function.

Also the gradient of the tanh function is **much steeper** as compared to the sigmoid function.

Rectified Linear Unit (ReLU)



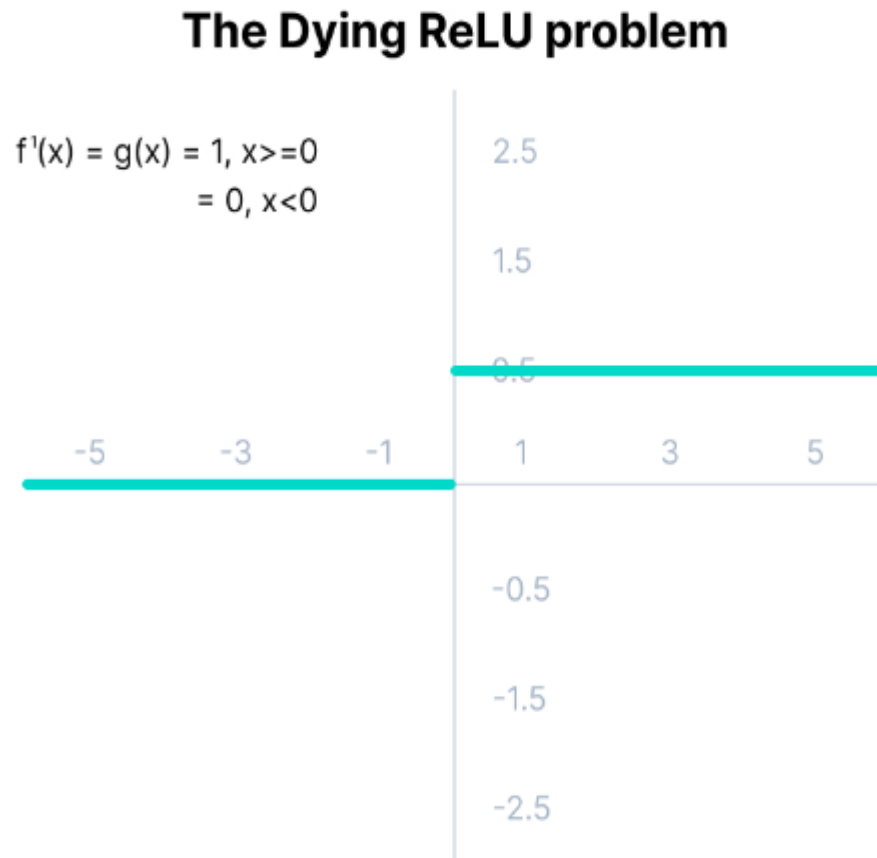
Mathematically it can be represented as:

$$f(x) = \max(0, x)$$

Advantages of ReLU function

- ❑ Since only a certain number of neurons are activated, the ReLU function is far **more computationally efficient** when compared to the sigmoid and tanh functions.
- ❑ ReLU **accelerates the convergence of gradient descent towards the global minimum** of the loss function due to its linear, non-saturating property.

The dying ReLU problem



- The negative side of the graph makes the gradient value zero. Due to this reason, during the backpropagation process, the weights and biases for some neurons are not updated. This can **create dead neurons** which never get activated.
- All the negative input values become zero immediately, which **decreases the model's ability** to fit or train from the data properly.

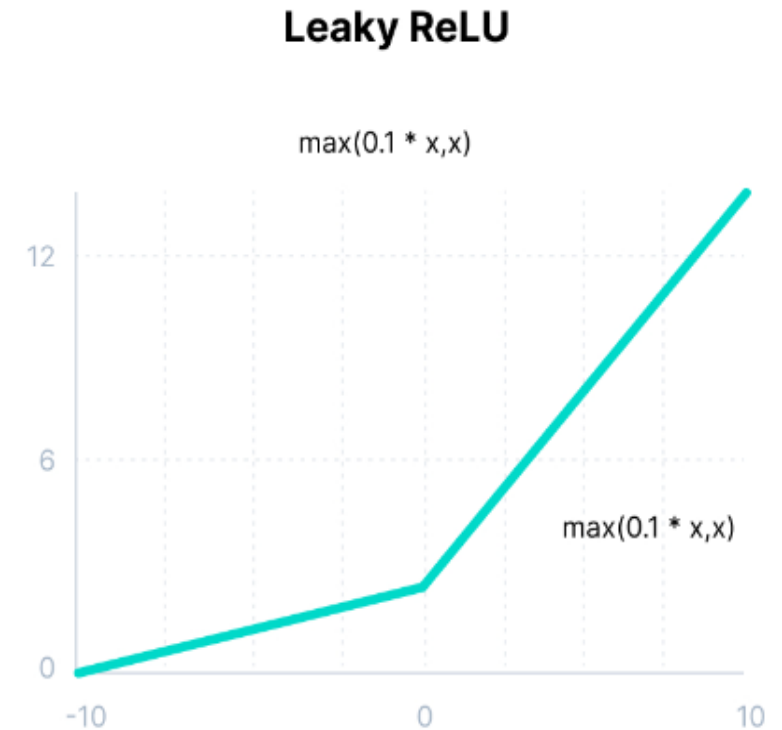
Leaky ReLU function

an improved version of ReLU function

to solve the Dying ReLU problem

as it has a small positive slope in the negative area.

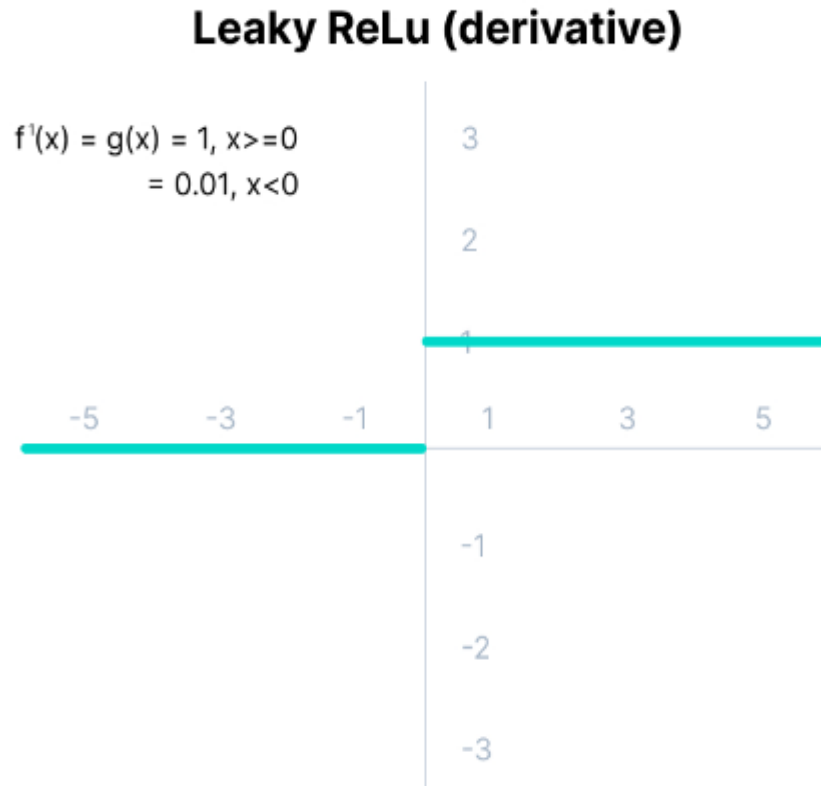
$$f(x) = \max(0.1x, x)$$



Advantages of Leaky ReLU function

- ❑ it does enable backpropagation, even for negative input values.
- ❑ By making this minor modification for negative input values, the gradient of the left side of the graph comes out to be a non-zero value. Therefore, we would no longer encounter dead neurons in that region.

Limitation of Leaky ReLU function



❑ The predictions may not be consistent for negative input values.

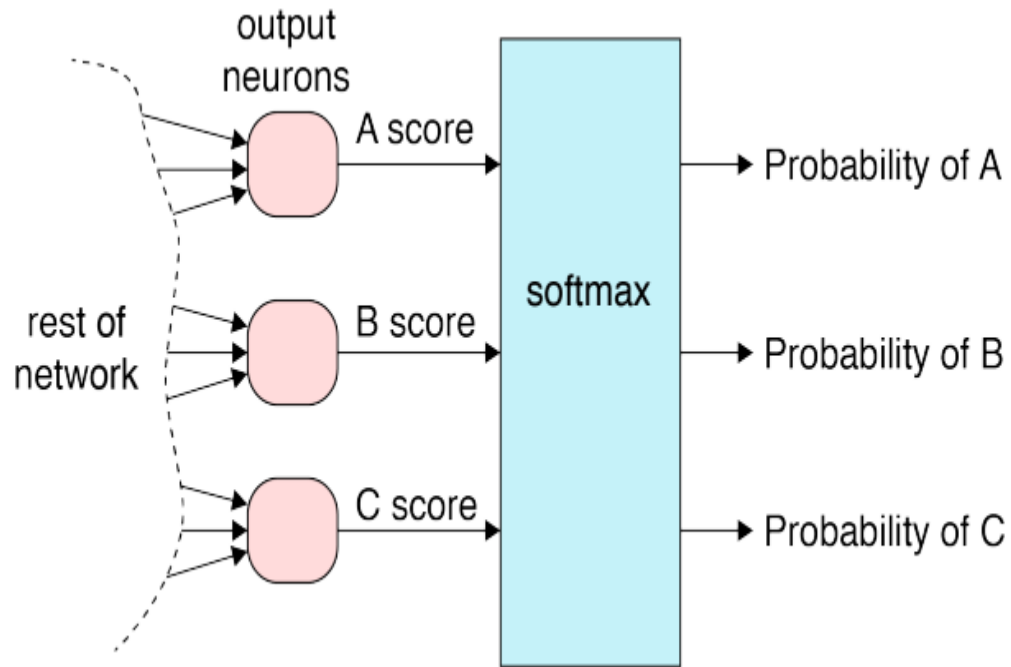
❑ The gradient for negative values is a small value that makes the learning of model parameters time-consuming.

Softmax function

- There's another kind of operation that we typically apply **only at the output neurons of a classifier** neural network, and even then only if there are 2 or more such neurons.
- It's not an activation function in the sense we've been using the term, because it **applies simultaneously to all the output neurons**, not just one.

Softmax function

- Softmax function is often described as a combination of multiple sigmoids.



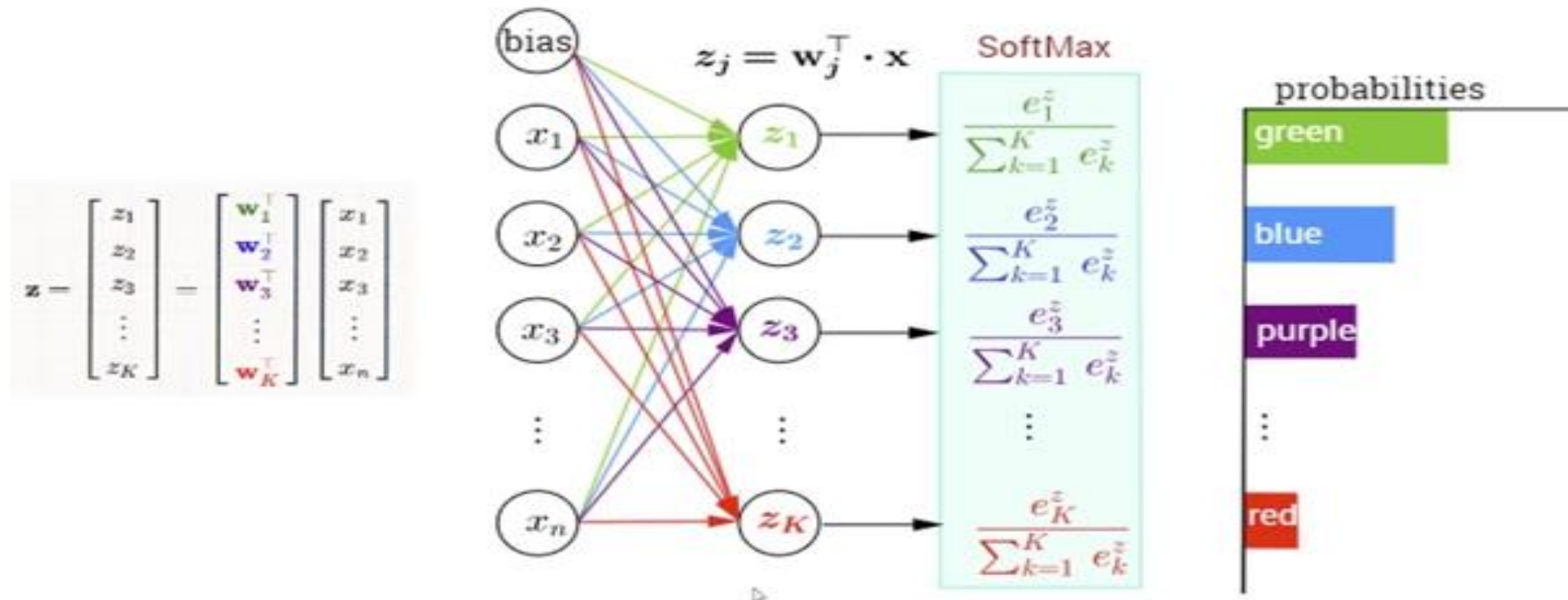
The softmax function takes all the network's outputs and modifies them simultaneously.

The result is that the scores are turned into **probabilities**.

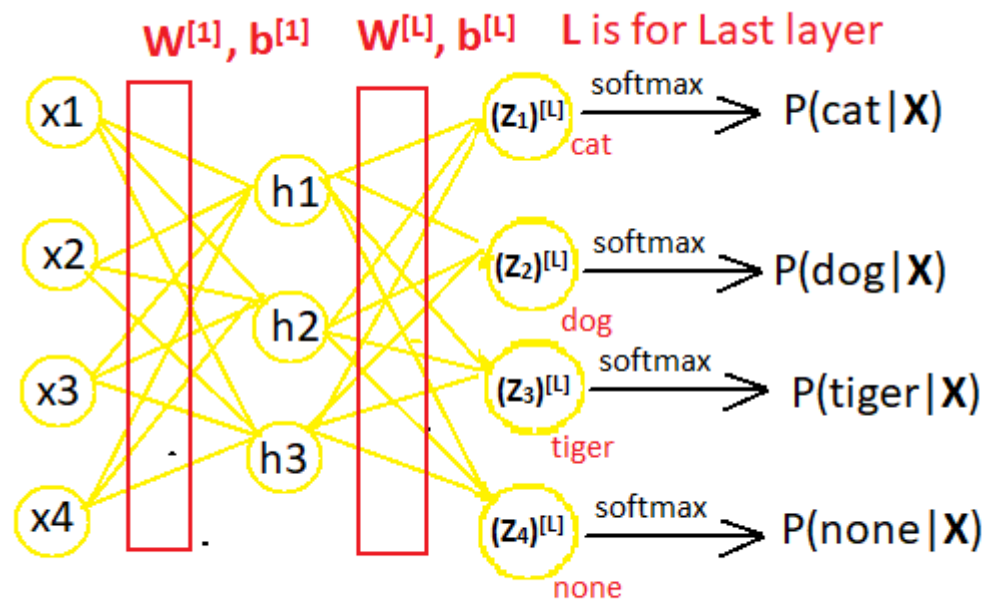
It is most commonly used as an activation function for the last layer of the neural network in the case of **multi-class classification**.

Softmax function

$$\text{softmax}(z_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$



Softmax function – An example



$$P(\text{cat}|\mathbf{X}) = \frac{e^{z_1^{[L]}}}{\sum e^{z^{[L]}}}$$

$$P(\text{dog}|\mathbf{X}) = \frac{e^{z_2^{[L]}}}{\sum e^{z^{[L]}}}$$

$$P(\text{tiger}|\mathbf{X}) = \frac{e^{z_3^{[L]}}}{\sum e^{z^{[L]}}}$$

$$P(\text{none}|\mathbf{X}) = \frac{e^{z_4^{[L]}}}{\sum e^{z^{[L]}}}$$

Softmax function – An example

$$Z^{[L]} = \begin{bmatrix} Z_1^{[L]} = 1.25 \\ Z_2^{[L]} = 2.44 \\ Z_3^{[L]} = 0.78 \\ Z_4^{[L]} = 0.12 \end{bmatrix}$$

$$e^{Z_1^{[L]}} = e^{1.25} = 3.49,$$

$$e^{Z_2^{[L]}} = e^{2.44} = 11.47,$$

$$e^{Z_3^{[L]}} = e^{0.78} = 2.18,$$

$$e^{Z_4^{[L]}} = e^{0.12} = 1.27.$$

$$\begin{aligned} \sum e^{Z^{[L]}} &= e^{Z_1^{[L]}} + e^{Z_2^{[L]}} + e^{Z_3^{[L]}} + e^{Z_4^{[L]}} \\ &= 3.49 + 11.47 + 2.18 + 1.27 = 18.41 \end{aligned}$$

$$P(cat|X) = \frac{e^{Z_1^{[L]}}}{\sum e^{Z^{[L]}}} = \frac{3.49}{18.41} = 0.189$$

$$P(dog|X) = \frac{e^{Z_2^{[L]}}}{\sum e^{Z^{[L]}}} = \frac{11.47}{18.41} = 0.623$$

$$P(tiger|X) = \frac{e^{Z_3^{[L]}}}{\sum e^{Z^{[L]}}} = \frac{2.18}{18.41} = 0.1184$$

$$P(none|X) = \frac{e^{Z_4^{[L]}}}{\sum e^{Z^{[L]}}} = \frac{1.27}{18.41} = 0.0689$$

Choosing the right Activation Function

- Sigmoid functions and their combinations generally work better in the case of classifiers
- Sigmoid/Logistic and Tanh functions should not be used in hidden layers avoided due to the vanishing gradient problem
- If we encounter a case of dead neurons in our networks the leaky ReLU function is the best choice
- Always keep in mind that ReLU function should only be used in the hidden layers
- As a rule of thumb, **you can begin with using ReLU function and then move over to other activation functions in case ReLU doesn't provide with optimum results**

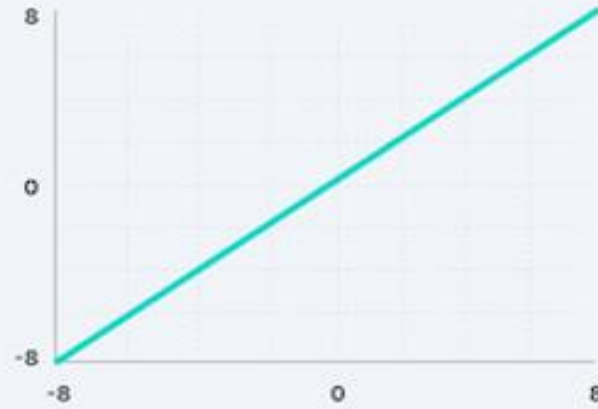
Choosing the right Activation Function

- rules for choosing the activation function for output layer based on the type of prediction problem that you are solving:
 - **Regression** - Linear Activation Function
 - **Binary Classification**—Sigmoid/Logistic Activation Function
 - **Multiclass Classification**—Softmax
 - **Multilabel Classification**—Sigmoid
- The activation function used in hidden layers is typically chosen based on the type of neural network architecture.
 - **Convolutional Neural Network (CNN)**: ReLU activation function.
 - **Recurrent Neural Network**: Tanh and/or Sigmoid activation function.

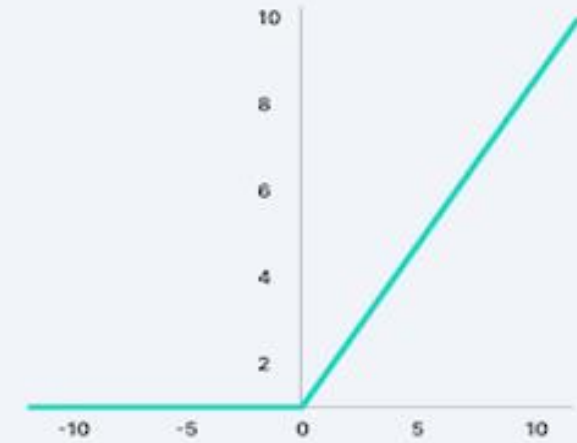
Binary Step Function



Linear



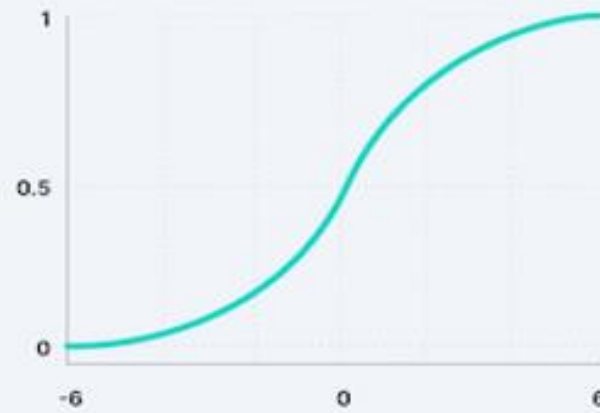
ReLU



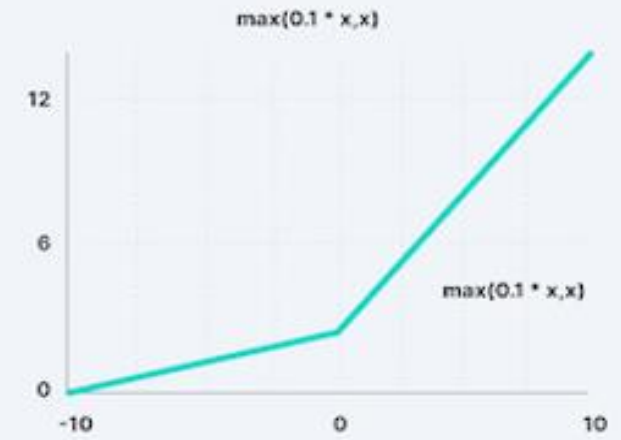
Tanh



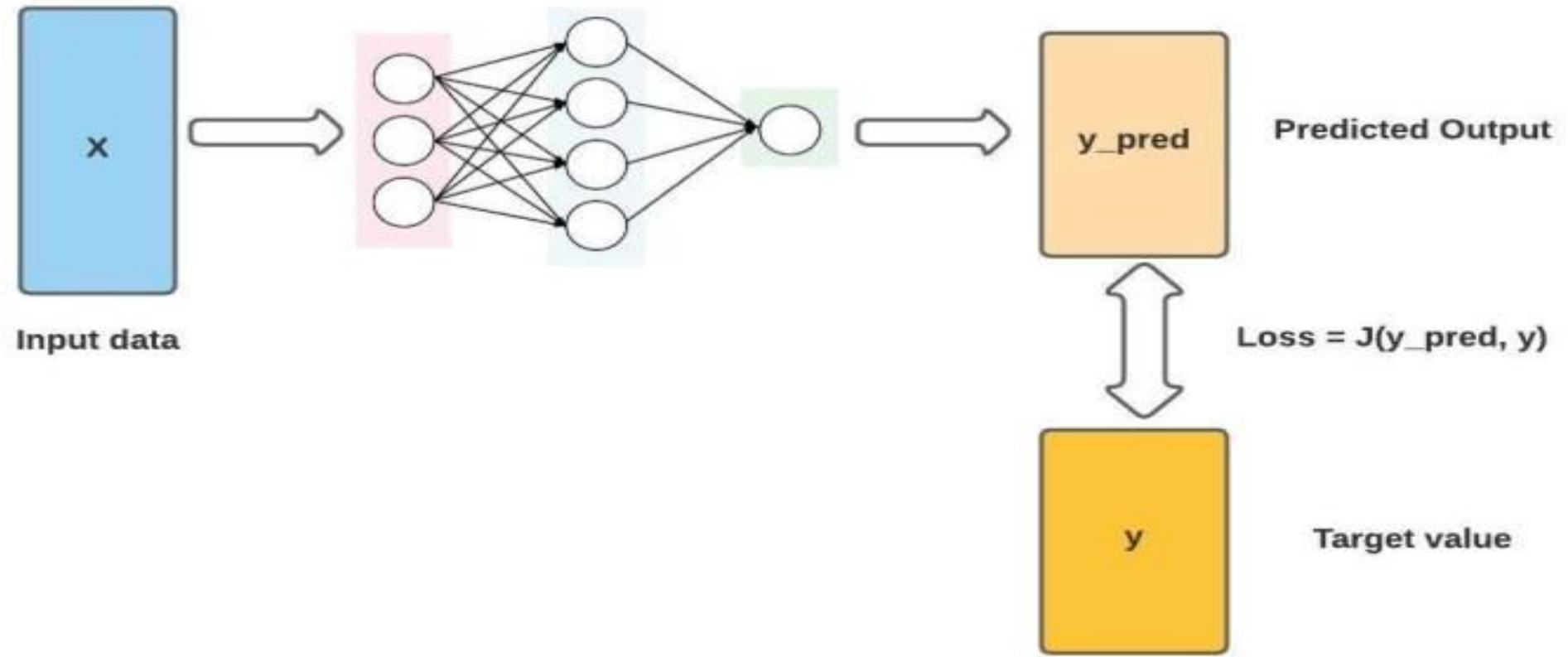
Sigmoid / Logistic



Leaky ReLU



Loss function



Loss function

- ❑ A **loss function** is a function that **compares** the target and predicted output values
- ❑ measures how well the neural network models the training data.
- ❑ This loss essentially tells something about the performance of the network:
the higher it is, the worse network performs overall.
- ❑ When training, we aim to minimize this loss between the predicted and target outputs.

Types of Loss function

REGRESSION LOSS FUNCTION

- if the problem is regression like predicting the continuous values
- Suppose if need to predict the weather conditions
- Ex. Mean Squared Error, Mean Absolute Error

CLASSIFICATION LOSS FUNCTION

- ❑ Classification loss is the case where the aim is to predict the output from the different categorical values
- ❑ if we have a dataset of handwritten images and the digit is to be predicted that lies between (0–9)
- ❑ Ex. Binary Cross-Entropy, Categorical Cross-Entropy

Mean Squared Error (MSE)

- ❑ Mean Squared Error (MSE) also called L2 Loss is also a loss function used for regression.
- ❑ It represents the difference between the original and predicted values extracted by squared the average difference over the data set.

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y})^2$$

Mean Squared Error (MSE)

- Use MSE when doing regression, believing that your target, conditioned on the input, is normally distributed, and want large errors to be significantly (quadratically) more penalized than small ones.
- MSE is sensitive towards outliers and given several examples with the same input feature values, the optimal prediction will be their mean target value.

Entropy – Information Theory Basics

- ❑ Information theory deals with the problem of encoding, decoding, transmitting, and manipulating information.
- ❑ The central idea in information theory is to quantify the amount of information contained in data.
- ❑ If it is always easy for us to predict the next token, then this data is easy to compress.
- ❑ However if we cannot perfectly predict every event, then we might sometimes be surprised.
- ❑ Our surprise is greater when we assigned an event lower probability.
- ❑ Claude Shannon settled on $\log (1/P (j)) = - \log P(j)$

to quantify one's surprisal at observing an event j having assigned it a (subjective) probability $P(j)$.

- ❑ The entropy is then the expected surprisal when one assigned the correct probabilities that truly match the data-generating process.

Cross Entropy

- we can think of the cross-entropy classification objective in two ways:
 - (i) as **maximizing the likelihood of the observed data**; and
 - (ii) as **minimizing our surprisal** (and thus the number of bits) required to communicate the labels.

Cross Entropy

- ❑ Cross-Entropy loss is also called logarithmic loss, log loss, or logistic loss.
- ❑ Each predicted class probability is compared to the actual class desired output 0 or 1
- ❑ and a score/loss is calculated that penalizes the probability based on how far it is from the actual expected value.
- ❑ The penalty is logarithmic in nature
- ❑ yielding a large score for large differences close to 1 and small score for small differences tending to 0.

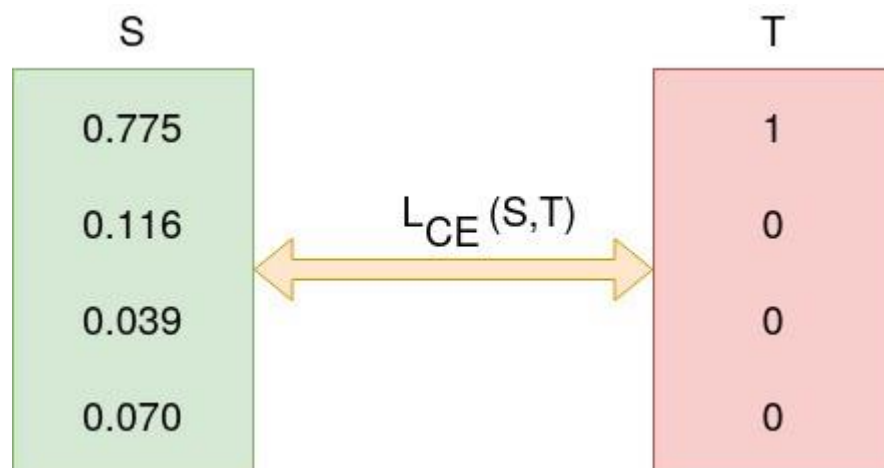
Cross Entropy

$$I(p, q) = -\sum p(x) \log q(x)$$

$p(x)$ is the probability distribution of “true” label and
 $q(x)$ depicts the estimation

Cross Entropy – An example

- Consider the classification problem with the probabilities (S) and the labels (T).
- The objective is to calculate for cross-entropy loss given these information.



$$\begin{aligned} L_{CE} &= - \sum_{i=1} T_i \log(S_i) \\ &= - [1 \log_2(0.775) + 0 \log_2(0.126) + 0 \log_2(0.039) + 0 \log_2(0.070)] \\ &= - \log_2(0.775) \\ &= 0.3677 \end{aligned}$$

S- Logits

T- one-hot encoded truth label

LCE - Cross-Entropy loss function

Activation function & Loss function

Type of Problem	Activation Function	Loss Function
Regression	Linear	Mean squared error
Binary Classification	Sigmoid	Binary cross entropy
Multi Class Classification	Softmax	Categorical cross entropy

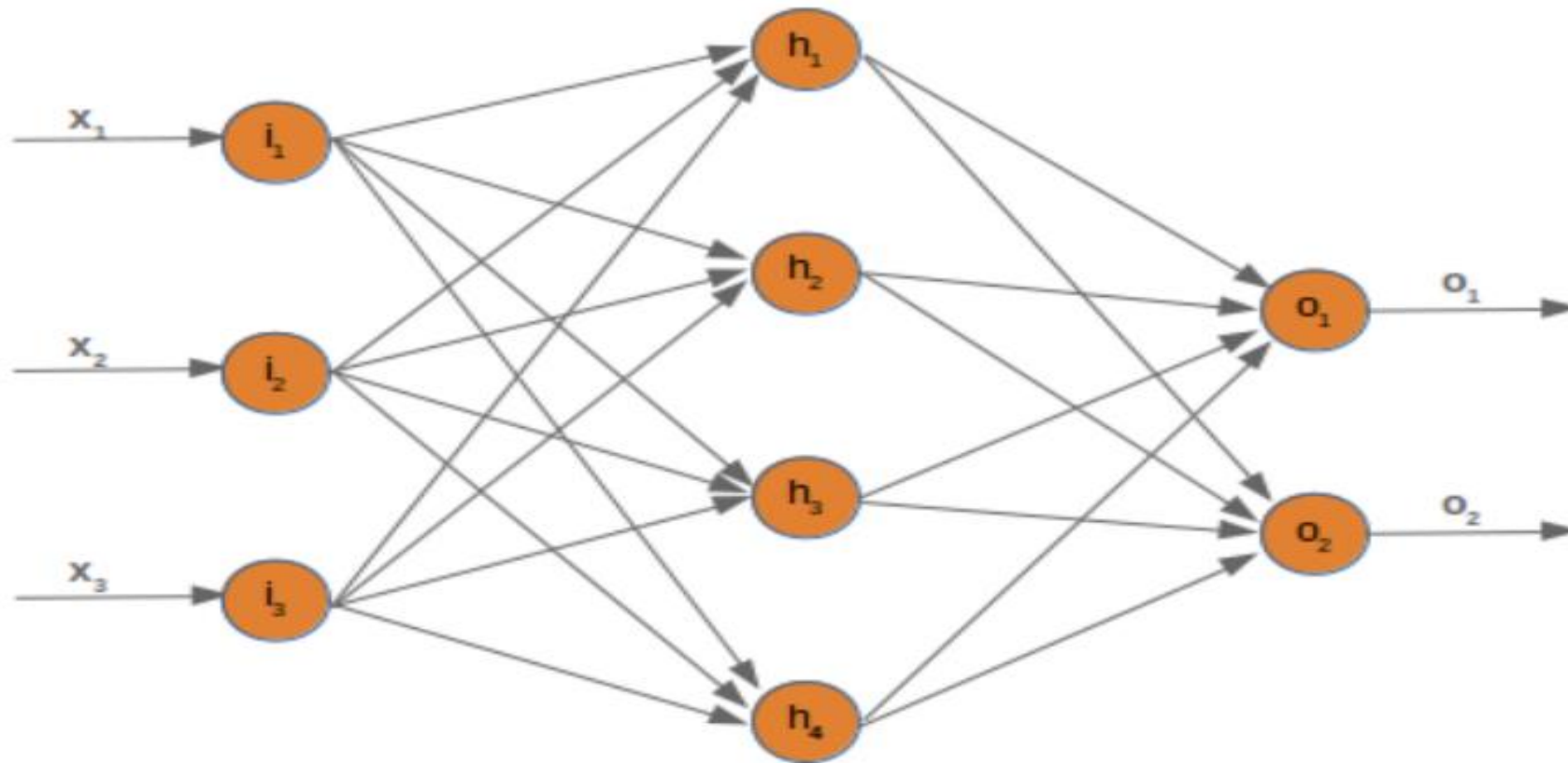


“

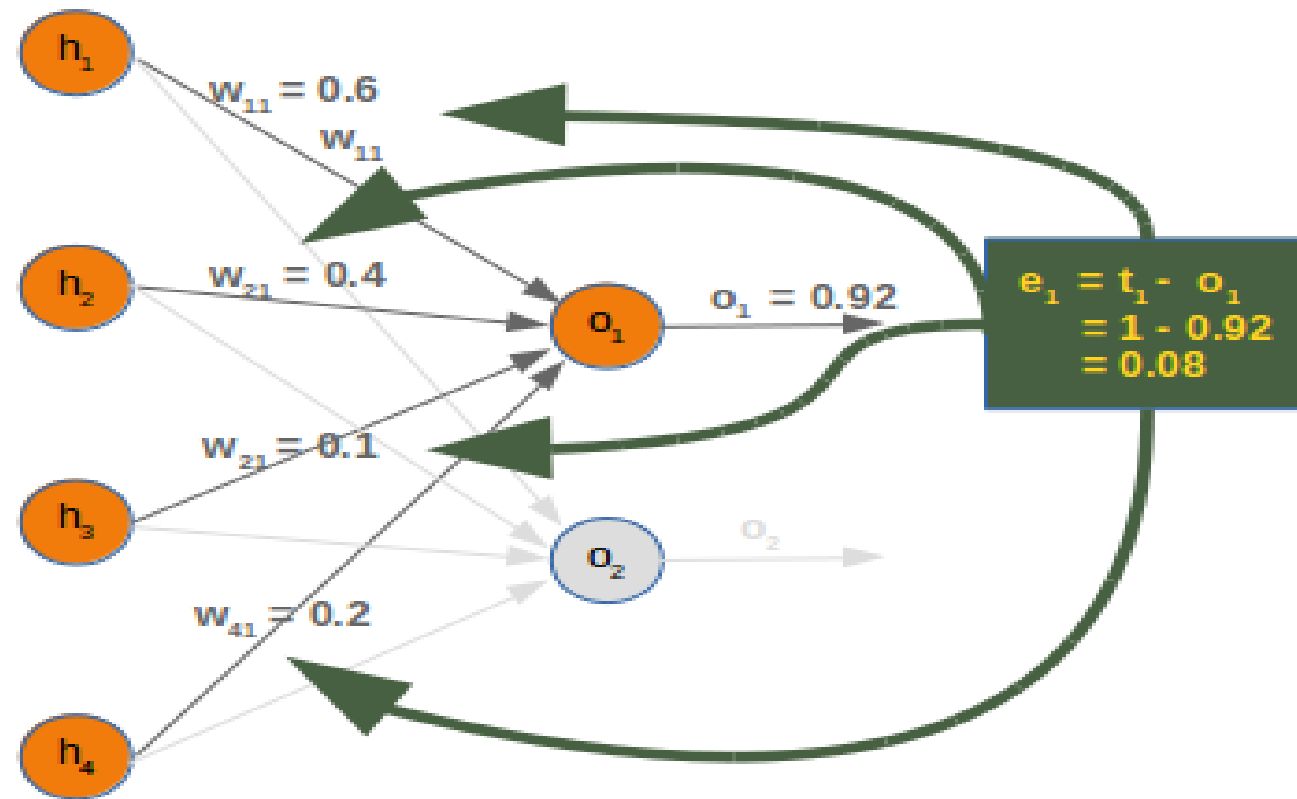
**We have the ingredients.
How do we make the
cake?**

Optimization – Learning with backpropagation

Training of NN



Training of NN



Training of NN

The error is the difference between the target and the actual output:

$$e_i = t_i - o_i$$

We will later use a squared error function, because it has better characteristics for the algorithm.

$$e_i = \frac{1}{2}(t_i - o_i)^2$$

In this case the error is

$$e_1 = t_1 - o_1 = 1 - 0.92 = 0.08$$

Updating weights & bias

- Depending on this error, we have to change the weights from the incoming values accordingly.
- Using some basic calculus, it is possible to derive a rule to update the weights on the connections coming into a neuron given a measure of the neuron's output error so as to reduce the neuron's error.
- The difficulty in training a neural network is that the weight-update rule requires an estimate of the error at a neuron, and although it is straightforward to calculate the error for each neuron in the output layer of the network, it is difficult to calculate the error for the neurons in the earlier layers.
- The standard way to train a neural network is to use an algorithm called the backpropagation algorithm to calculate the error for each neuron in the network and then use the weight-update rule to modify the weights in the network.

Backpropagation Algorithm

The main steps in the algorithm are as follows:

1. Calculate the error for the neurons in the output layer and use the weight-update rule to update the weights coming into these neurons.
2. Share the error calculated at a neuron with each of the neurons in the preceding layer that is connected to that neuron in proportion to the weight of the connection between the two neurons.
3. For each neuron in the preceding layer, calculate the overall error of the network that the neuron is responsible for by summing the errors that have been backpropagated to it and use the result of this error summation to update the weights on the connections coming into this neuron.
4. Work back through the rest of the layers in the network by repeating steps 2 and 3 until the weights between the input neurons and the first layer of hidden neurons have been updated.

How to update the weights on the connections??

1. Naïve Solution – expensive & inefficient
2. Gradient Based

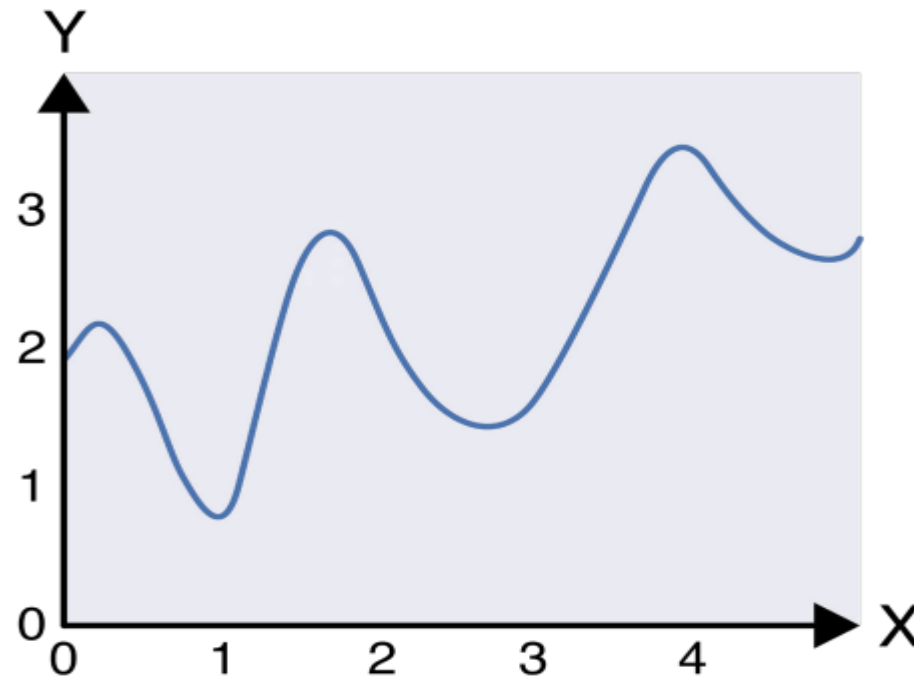
Gradient

- All operations used in network are “differentiable”
- Then compute gradient of loss with respect to network coefficients.
- The process to tie together the loss function and model parameters to update the network based on output of loss function is known as optimization.

Gradient

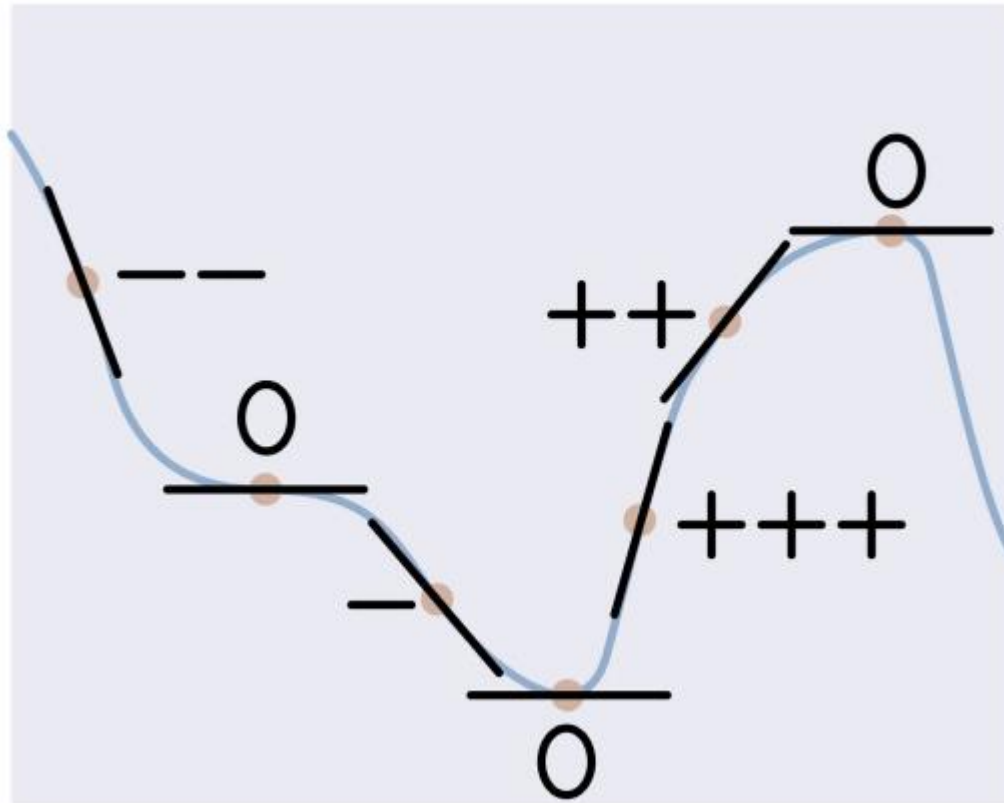
- ❑ We have to calculate what a small change in each individual weight would do to the loss function.
- ❑ So we adjust each parameter based on its gradient.
- ❑ It is a small step in the determined direction.
- ❑ A gradient is a degree of inclination.
- ❑ It facilitates the rate of change of one variable with respect to another variable.

Gradient



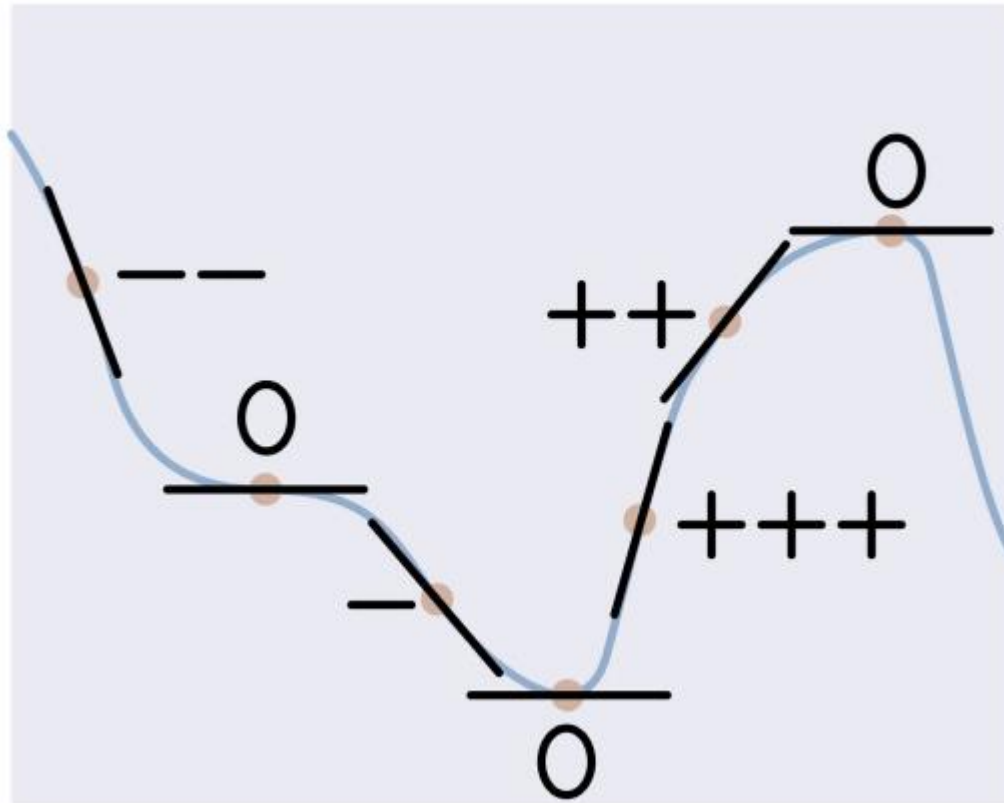
A curve in two dimensions. Values of X increase as we move right, and values of Y increase as we move up.

Gradient



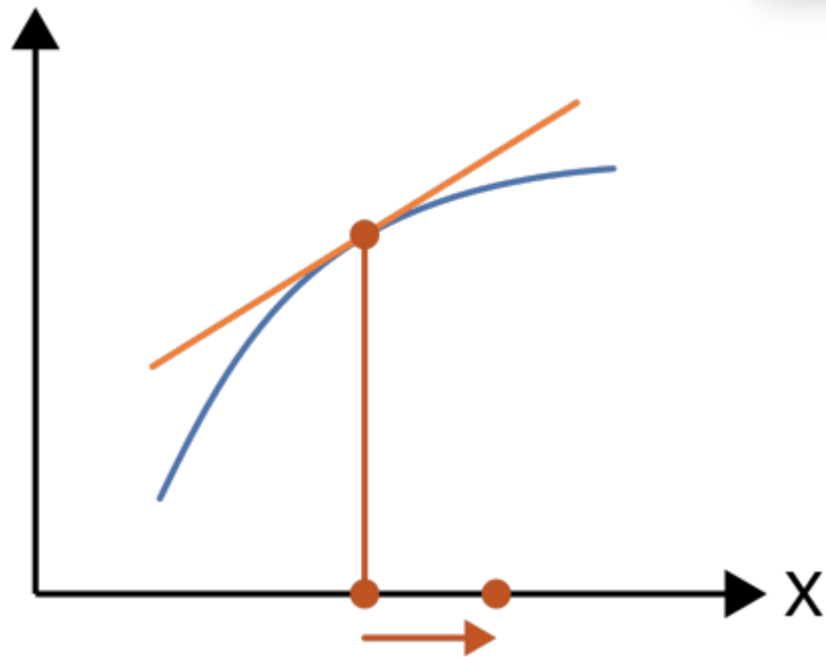
Marking the tangent lines by whether they have a positive slope (+), negative slope (–), or are flat (0). To indicate the range of slopes shown, the more positive or negative each slope, the more + or – signs we’ve drawn.

Gradient



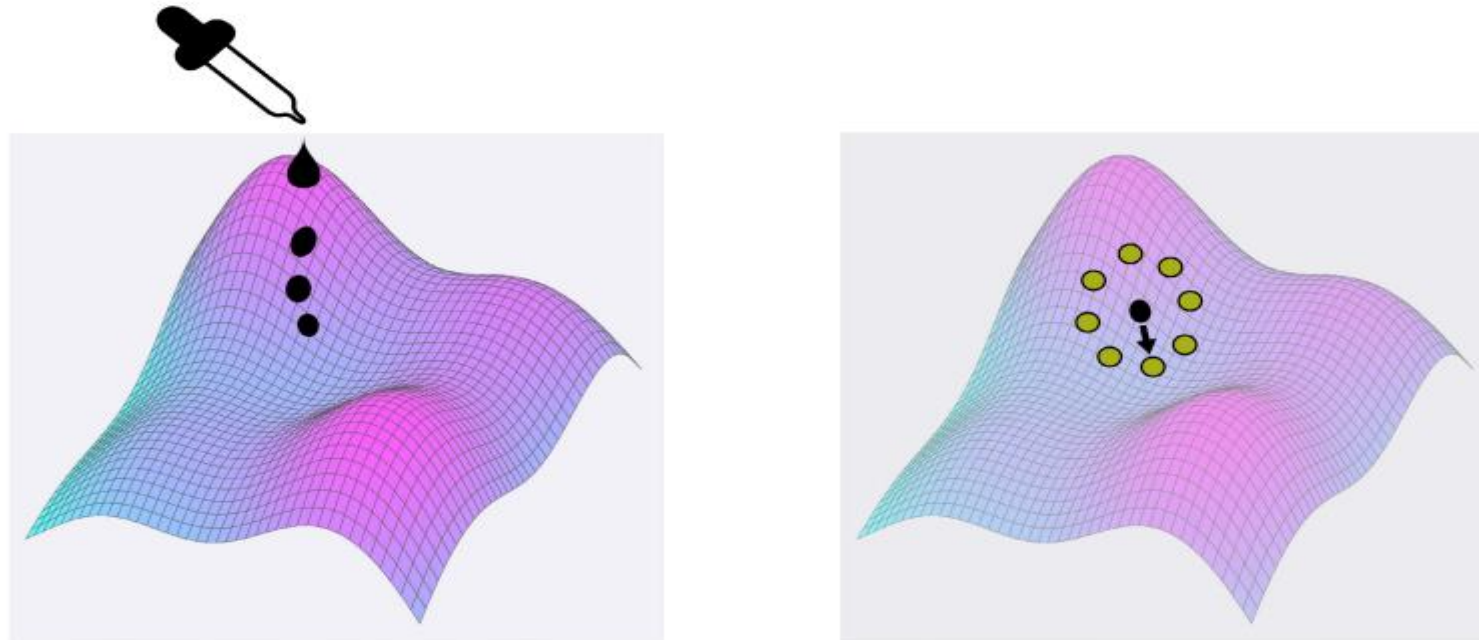
Marking the tangent lines by whether they have a positive slope (+), negative slope (–), or are flat (0). To indicate the range of slopes shown, the more positive or negative each slope, the more + or – signs we’ve drawn.

Derivative



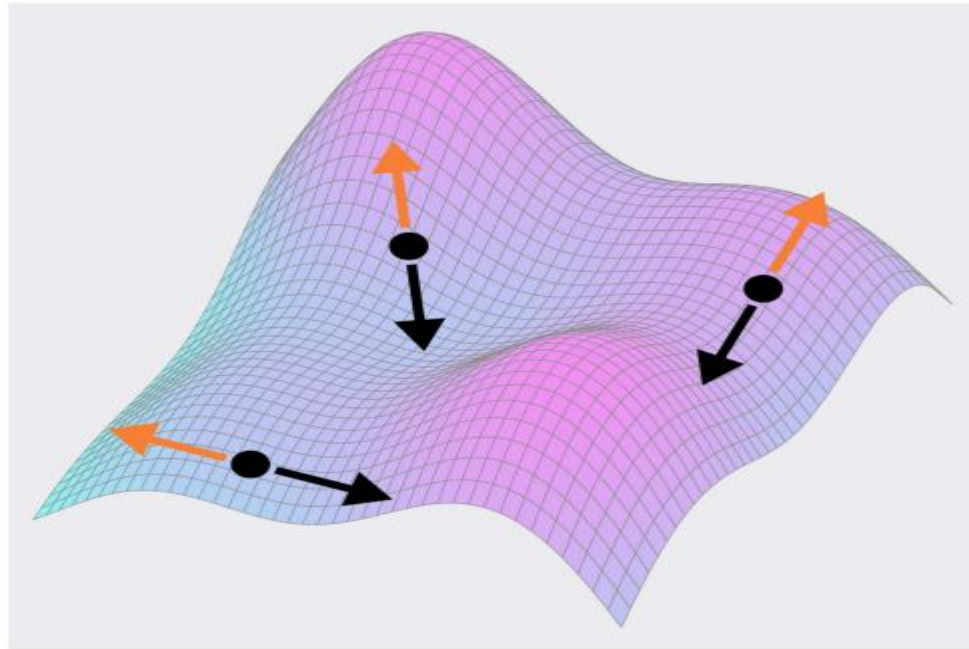
The derivative at a point tells us which way to move to find larger or smaller values of the curve. At the marked point, the derivative is positive, so if we take a step to the right (along positive X), we'll find a larger value of the curve.

Gradient



Out of all the ways to move, the water will always follow the steepest route downhill. The direction followed by the water is called the direction of maximum descent.

Gradient



The direction of maximum ascent (in orange) always points in exactly the opposite direction as maximum descent (in black).

Gradient

- We can then move the coefficient in opposite direction from gradient thus decreasing the loss.
- A gradient of a function is a vector of partial derivatives w.r.t. to all independent variables.
- It always points toward steepest increase in function.

Gradient

The list of popular gradients is enlisted as--

1. Gradient Descent
2. Stochastic Gradient Descent
3. Mini Batch Gradient Descent
4. AdaGrad
5. RMSProp
6. Adam
7. Momentum Based Gradient Descent
8. Nesterov Accelerated Gradient Descent

Gradient Descent – The grand daddy of Optimization

- It is iterative algorithm that starts off at a random point on loss function and travel down its slope in steps until it reaches the lowest point (minimum) of the function.

Thus, if ΔW_{ij} denoted the weight update of the link connecting the i -th and j -th neuron of the two neighboring layers then

$$\Delta W_{ij} = \eta \frac{\partial E}{\partial W_{ij}}$$

where η is the **learning rate parameter** and $\frac{\partial E}{\partial W_{ij}}$ is the **error gradient** with reference to the weight W_{ij}

Gradient Descent – The grand daddy of Optimization

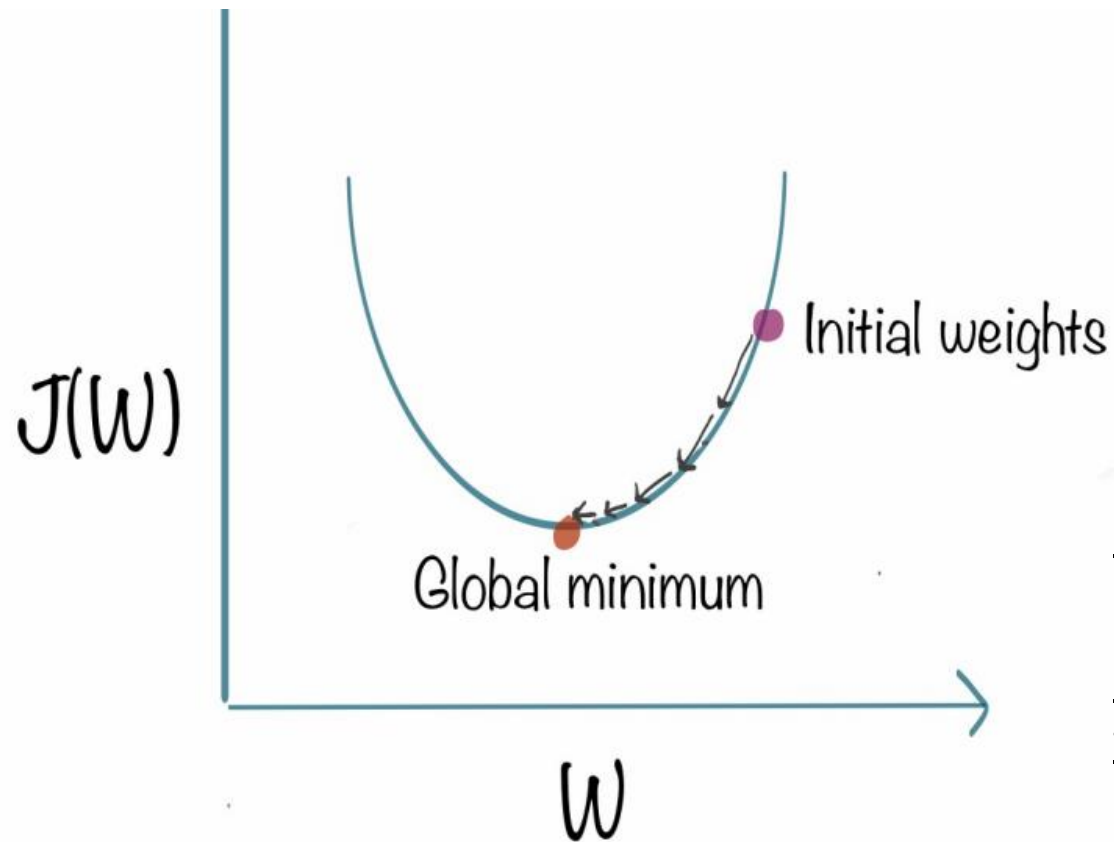
It is Gradient descent works as follows:

1. It starts with some coefficients, sees their cost, and searches for cost value lesser than what it is now.
2. It moves towards the lower weight and updates the value of the coefficients.
3. The process repeats until the local minimum is reached. A local minimum is a point beyond which it cannot proceed.

Gradient Descent – The grand daddy of Optimization

Gradient descent works best for most purposes.

However, it has some downsides too.



It is expensive to calculate the gradients if the size of the data is huge.

Gradient descent works well for convex functions but it doesn't know how far to travel along the gradient for nonconvex functions.

Stochastic Gradient Descent - SGD

- ❑ The term stochastic means randomness on which the algorithm is based upon.
- ❑ In stochastic gradient descent, instead of taking the whole dataset for each iteration, we randomly select the batches of data.
- ❑ That means we only take few samples from the dataset.

$$w := w - \eta \nabla Q_i(w).$$

- ❑ The procedure is first to select the initial parameters w and learning rate η . Then randomly shuffle the data at each iteration to reach an approximate minimum.

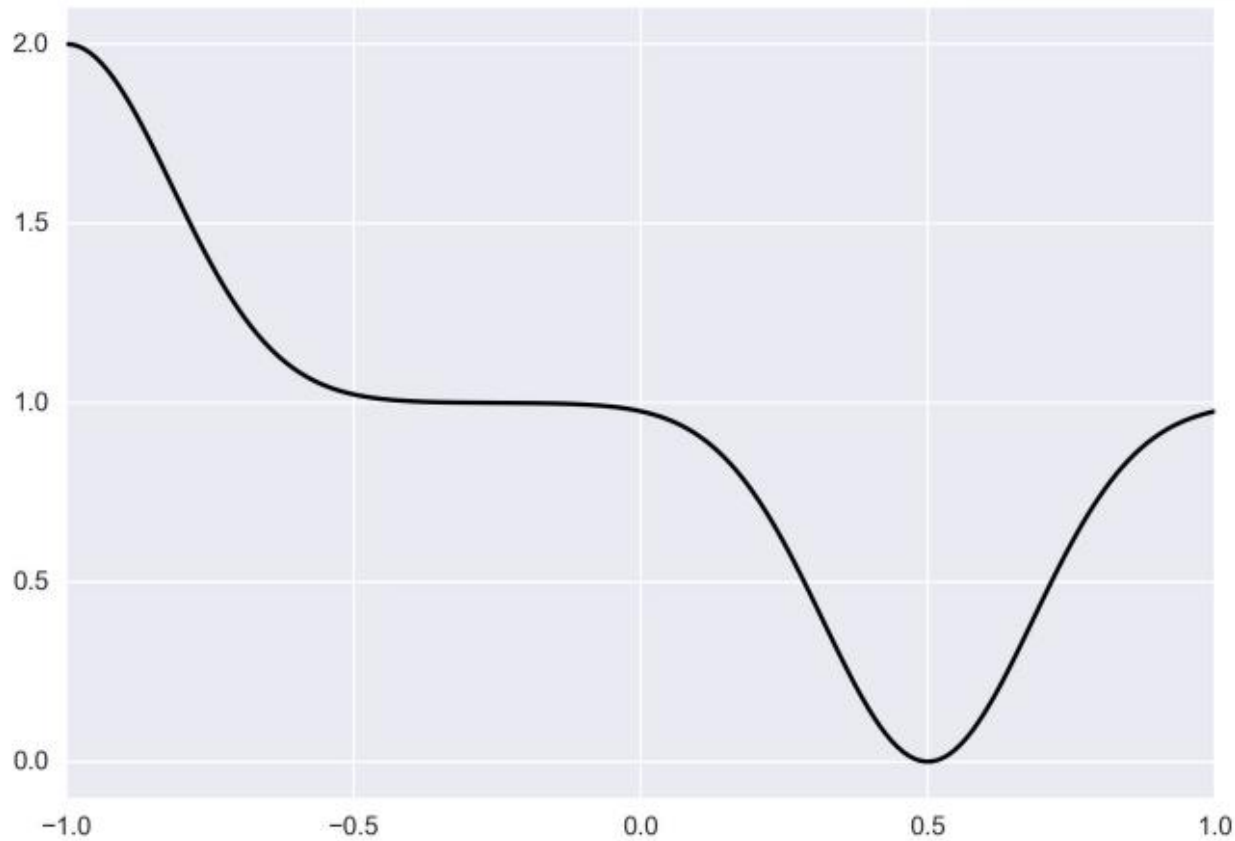
Stochastic Gradient Descent - SGD

- ❑ We call **SGD an online algorithm**, because it doesn't require the samples to be stored or even consistent from one epoch to the next.
- ❑ It just handles each sample as it arrives and updates the network immediately.
- ❑ Since we are not using the whole dataset but the batches of it for each iteration, the path took by the algorithm is **full of noise** as compared to the gradient descent algorithm.
- ❑ Thus, SGD uses a higher number of iterations to reach the local minima.
- ❑ Due to an increase in the number of iterations, the overall computation time increases.
- ❑ But even after increasing the number of iterations, the **computation cost is still less** than that of the gradient descent optimizer.

Mini Batch Gradient Descent

- ❑ We can find a nice middle ground between the extremes of batch gradient descent, which updates once per epoch, and stochastic gradient descent, which updates after every sample.
- ❑ **We update the weights after some fixed number of samples have been evaluated.**
- ❑ This number is almost always considerably smaller than the **batch size** (that is, the number of samples in the training set).
- ❑ a set of that many samples drawn from the training set is a **mini-batch**.
- ❑ The mini-batch size is frequently a power of 2 between about 32 and 256, and often chosen to fully use the parallel capabilities of our GPU, if we have one.

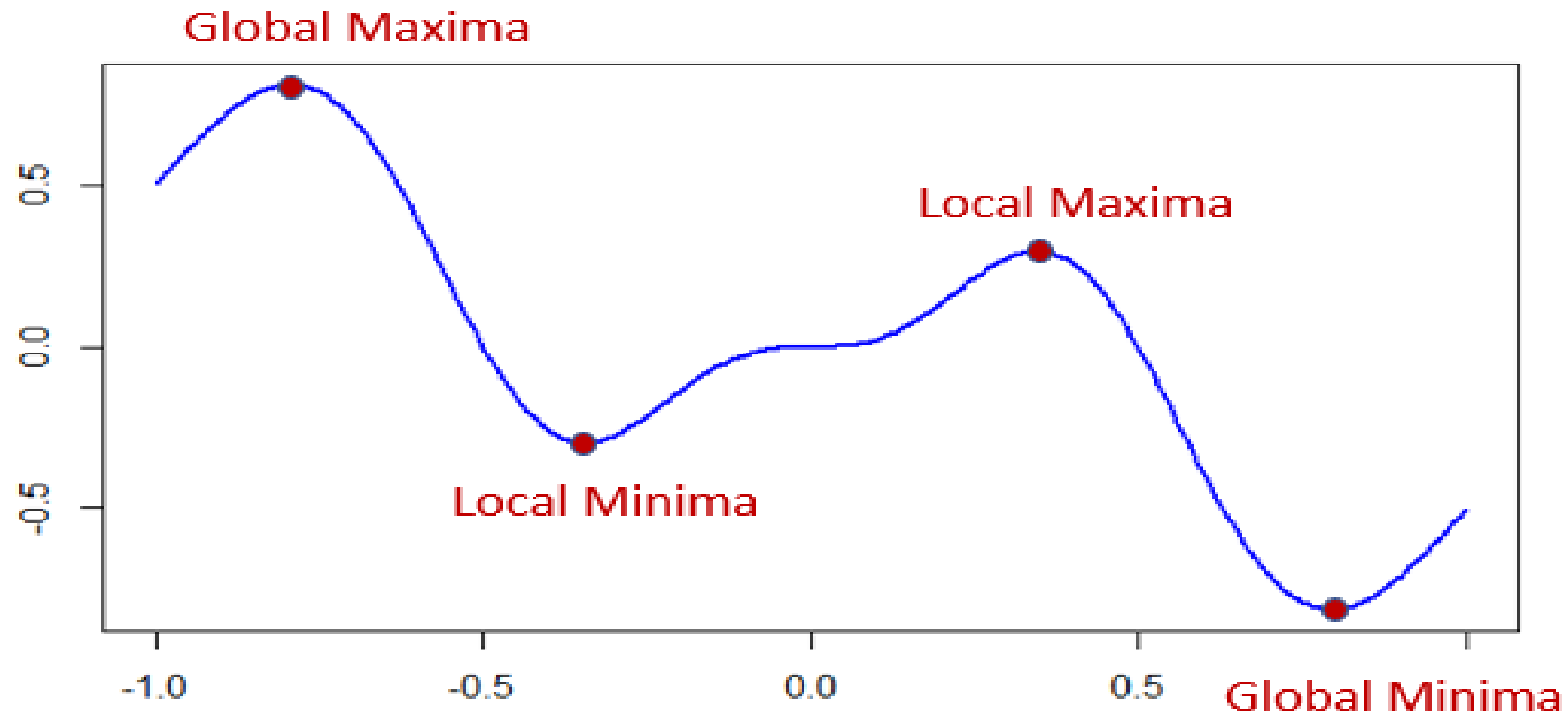
Momentum based GD



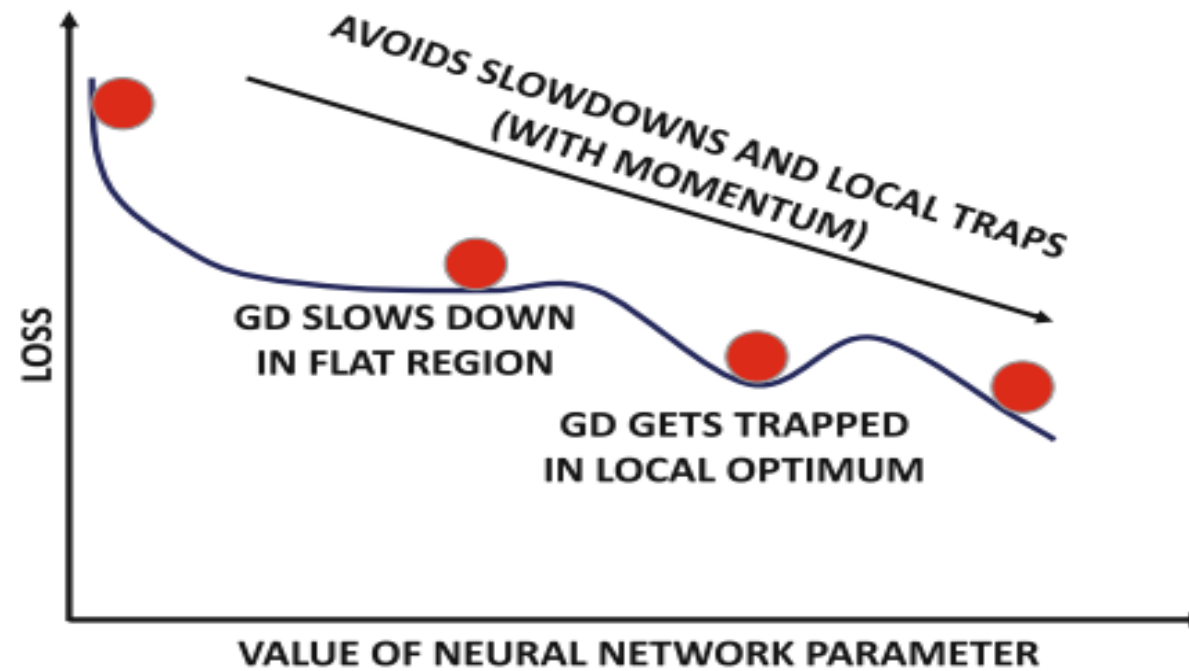
Suppose we're near the left side fig.
As we roll down the hill, we'll reach the plateau starting at around -0.5 .

With regular gradient descent,
we'd stop on the plateau since the gradient is zero

Momentum based GD

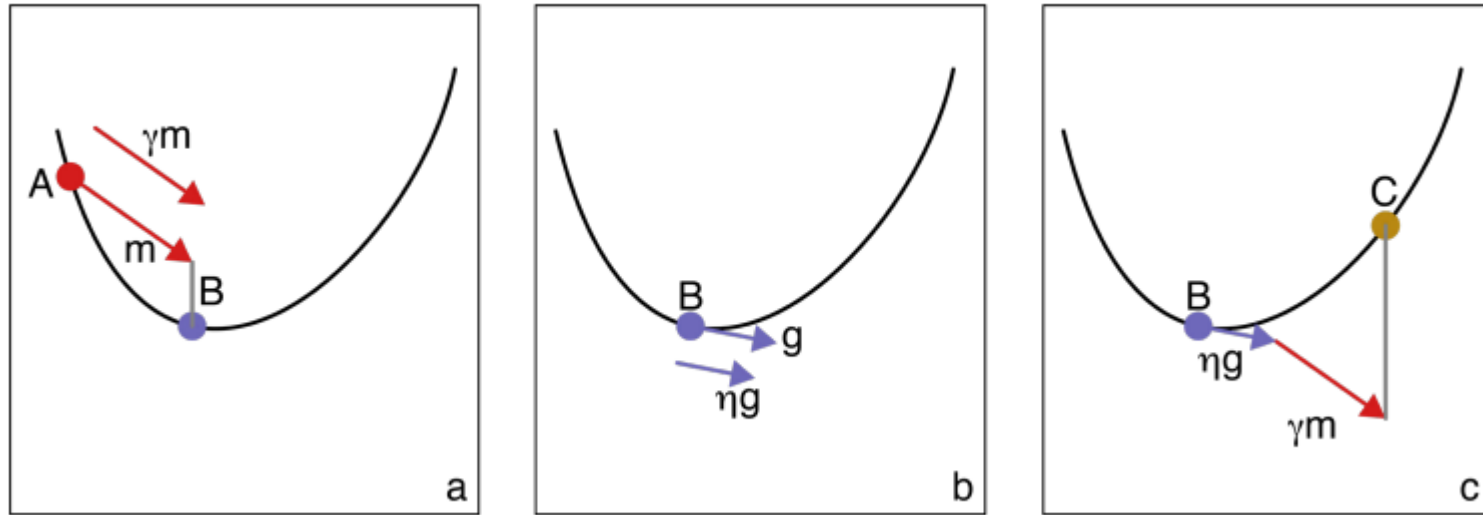


Momentum based GD



Momentum based GD

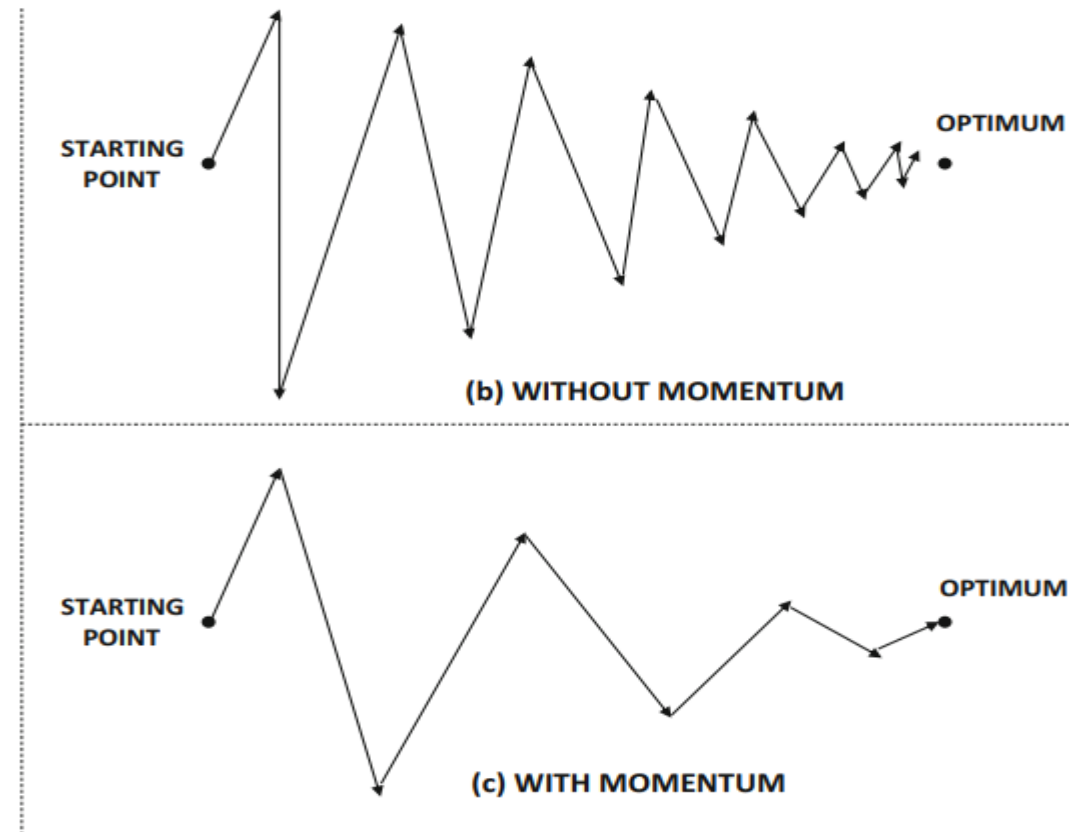
- ❑ If we could get our learning process to use this kind of momentum, that would be great, because it would help us cross over plateaus like the one in the figure.
- ❑ The technique of momentum gradient descent [Qian99] is based on this idea.
- ❑ For each step, once we calculate how much we want each weight to change, we then add in a small amount of its change from the previous step.
- ❑ So if the change on a given step is 0, or nearly 0, but we had some larger change on the last step, we'll use some of that prior motion now, pushing us along over the plateau.



Finding the step for gradient descent with momentum.

- (a) When we're at point B, we look back to the previous point A and find the "momentum," or the change we applied to point A. We call this " m ." This momentum is scaled by γ , a number from 0 to 1, giving us the shorter arrow labeled γm .
- (b) As with basic gradient descent, we find the gradient at B and scale it by η .
- (c) Starting at point B, we add the scaled gradient ηg and the scaled momentum γm to give us the new point C.

Momentum based GD – Accelerating speed of learning



Momentum based GD

- In order to understand this point, consider a setting in which one is performing gradient descent with respect to the parameter vector W .
- The normal updates for gradient-descent with respect to loss function L are as follows:

$$\bar{V} \leftarrow -\alpha \frac{\partial L}{\partial \bar{W}}; \quad \bar{W} \leftarrow \bar{W} + \bar{V}$$

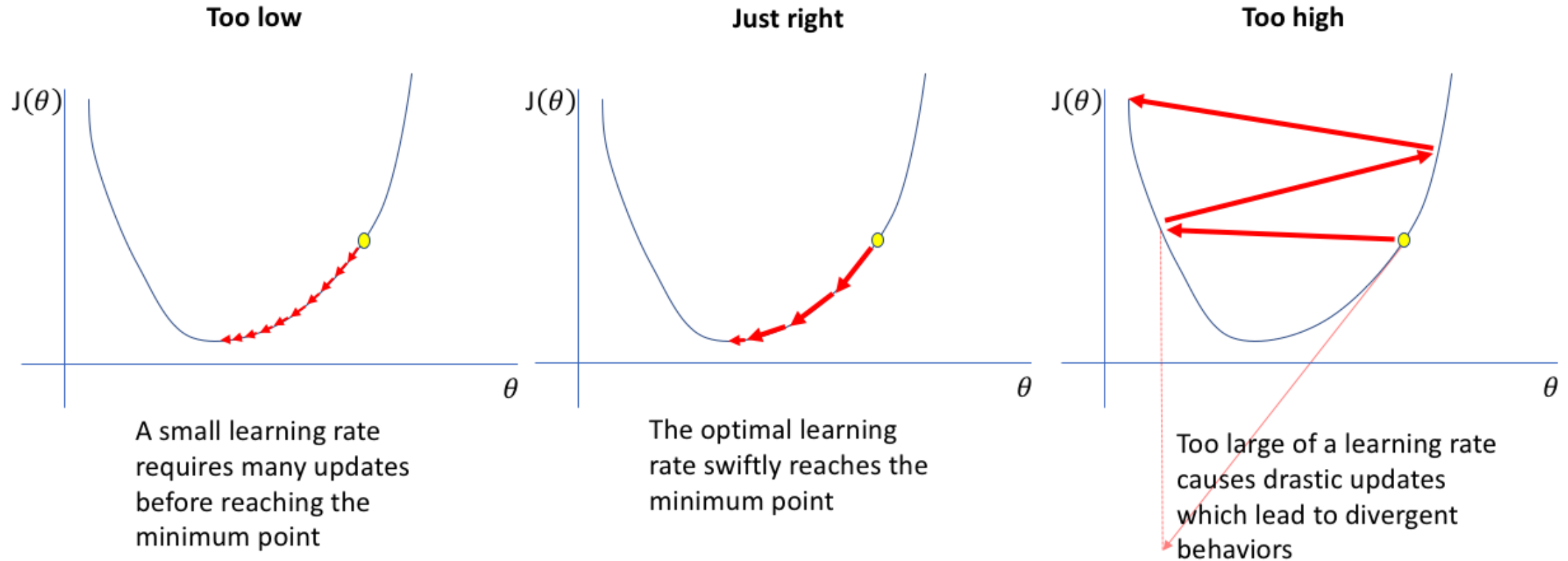
- Here, α is the learning rate. In momentum-based descent, the vector V is modified with exponential smoothing, where $\beta \in (0, 1)$ is a smoothing parameter:

$$\bar{V} \leftarrow \beta \bar{V} - \alpha \frac{\partial L}{\partial \bar{W}}; \quad \bar{W} \leftarrow \bar{W} + \bar{V}$$

Momentum based GD

- ❑ Larger values of β help the approach pick up a consistent velocity V in the correct direction.
- ❑ Setting $\beta = 0$ specializes to straightforward mini-batch gradient-descent.
- ❑ The parameter β is also referred to as the momentum parameter or the friction parameter. The word “friction” is derived from the fact that small values of β act as “brakes,” much like friction.
- ❑ The basic idea is to give greater preference to consistent directions over multiple steps, which have greater importance in the descent.
- ❑ This allows the use of larger steps in the correct direction without causing overflows or “explosions” in the sideways direction.
- ❑ As a result, learning is accelerated.

Proper learning rate



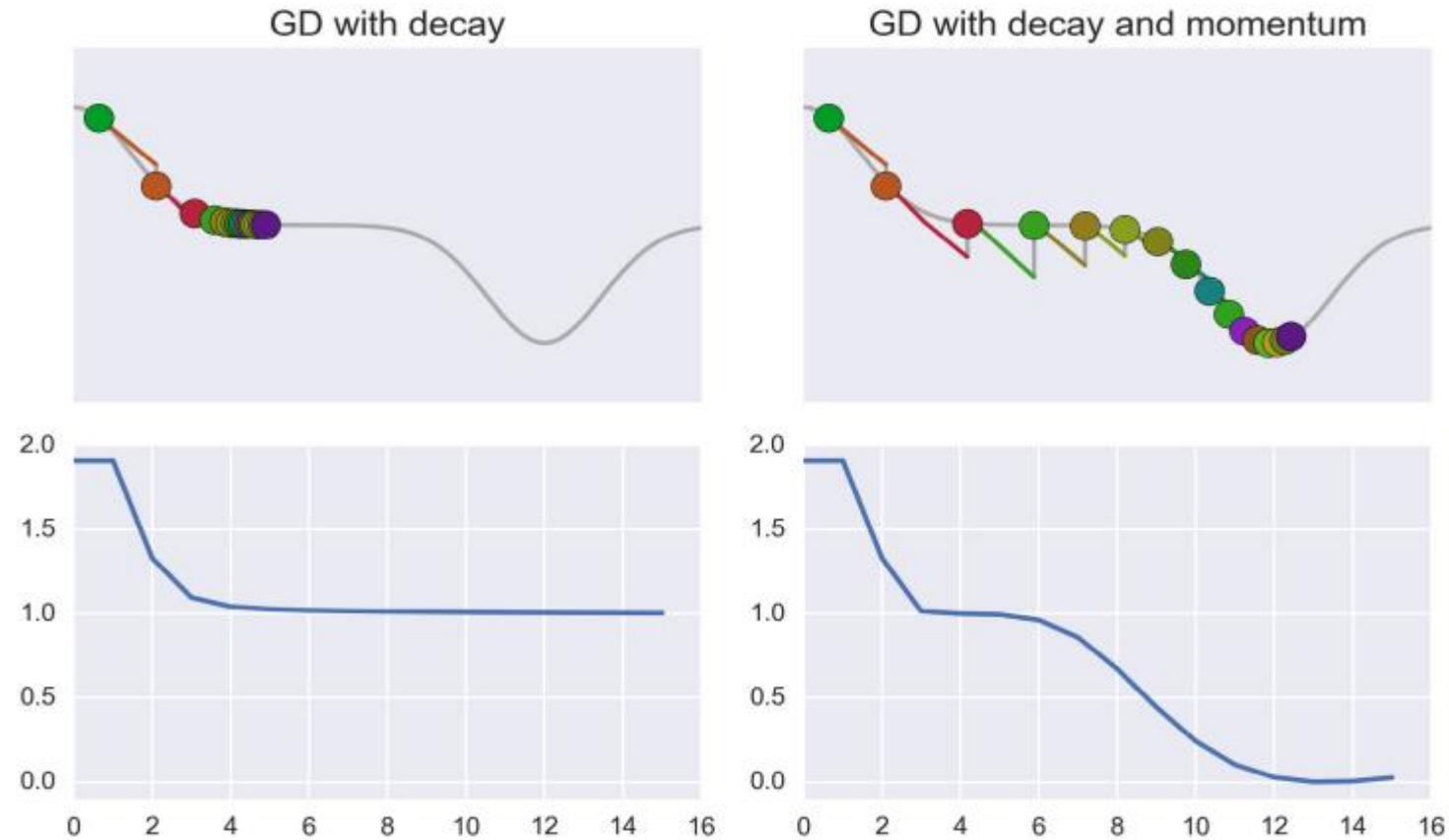
Learning Rate Decay

- ❑ A constant learning rate is not desirable because it poses a dilemma to the analyst.
- ❑ A lower learning rate used early on will cause the algorithm to take too long to come even close to an optimal solution.
- ❑ On the other hand, a large initial learning rate will allow the algorithm to come reasonably close to a good solution at first; however, the algorithm will then oscillate around the point for a very long time, or diverge in an unstable way, if the high rate of learning is maintained.
- ❑ In either case, maintaining a constant learning rate is not ideal.
- ❑ Allowing the learning rate to decay over time can naturally achieve the desired learning-rate adjustment to avoid these challenges.

Momentum based GD

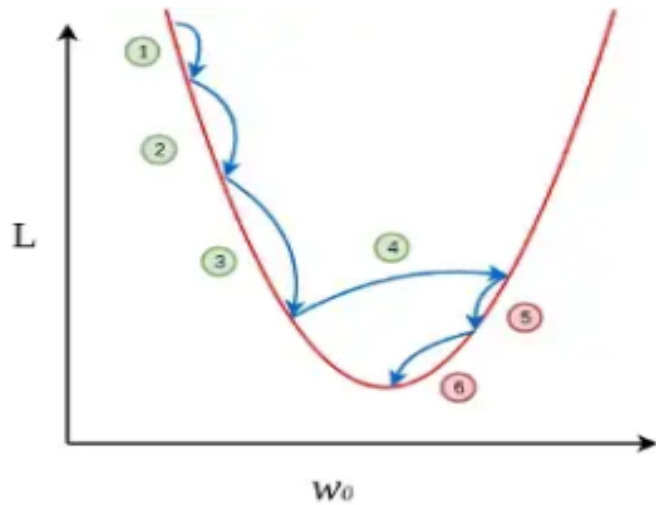
The ball rolls for a little while over the plateau, losing energy and slowing.

But it reaches the next valley before it comes to a halt, giving it the chance to drop into the valley and get down to the minimum error.



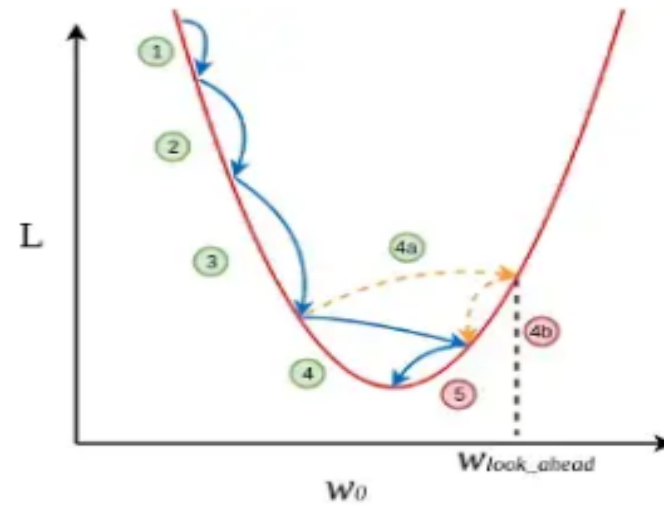
Nesterov Accelerated Gradient (NAG)

“look before you leap”



(a) Momentum-Based Gradient Descent

$$\text{Green Circle} \Rightarrow \frac{\partial L}{\partial w_0} = \frac{\text{Negative}(-)}{\text{Positive}(+)}$$



(b) Nesterov Accelerated Gradient Descent

$$\text{Red Circle} \Rightarrow \frac{\partial L}{\partial w_0} = \frac{\text{Negative}(-)}{\text{Negative}(-)}$$

NAG = Momentum
+
GD with decay

Fact Analysis

- The frequent parameters will start receiving very small update
- Therefore decay learning rate for parameters in proportion to their update history.

Intuition

Decay the learning rate for parameters in proportion to their update history

(more updates means more decay).

AdaGrad — Adaptive Gradient Algorithm

$$v_t^w = v_{t-1}^w + (\nabla w_t)^2$$
$$w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t^w + \epsilon}} * \nabla w_t$$

$$v_t^b = v_{t-1}^b + (\nabla b_t)^2$$
$$b_{t+1} = b_t - \frac{\eta}{\sqrt{v_t^b + \epsilon}} * \nabla b_t$$

It is clear from the update rule that history of the gradient is accumulated in $\mathbf{v}$.

The smaller the gradient accumulated, the smaller the $\mathbf{v}$ value will be, leading to a bigger learning rate (because $\mathbf{v}$ divides η).

AdaGrad — Adaptive Gradient Algorithm

By using a parameter specific learning rate AdaGrad ensures that despite sparsity $\mathbf{w}$ gets a higher learning rate and hence larger updates.

Furthermore, it also ensures that if $\mathbf{b}$ undergoes a lot of updates, its effective learning rate decreases because of the growing denominator.

BUT

In practice, Over time the effective learning rate for $\mathbf{b}$ will decay to an extent that there will be no further updates to $\mathbf{b}$.

Can we avoid this? RMSProp can!

RMSProp — Root Mean Square Propagation

Intuition

AdaGrad decays the learning rate very aggressively (as the denominator grows). As a result, after a while, the frequent parameters will start receiving very small updates because of the decayed learning rate. To avoid this why not decay the denominator and prevent its rapid growth.

$$v_t^w = \beta * v_{t-1}^w + (1 - \beta)(\nabla w_t)^2$$
$$w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t^w + \epsilon}} * \nabla w_t$$

$$v_t^b = \beta * v_{t-1}^b + (1 - \beta)(\nabla b_t)^2$$
$$b_{t+1} = b_t - \frac{\eta}{\sqrt{v_t^b + \epsilon}} * \nabla b_t$$

Adam — Adaptive Moment Estimation

In addition to that, use a cumulative history of gradients.

$$m_t = \beta_1 * m_{t-1} + (1 - \beta_1) * \nabla w_t$$

$$v_t = \beta_2 * v_{t-1} + (1 - \beta_2) * (\nabla w_t)^2$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_t + \epsilon}} * \hat{m}_t$$

Training FF DNN- Process

- The first input is fed to the network, which is represented as matrix x_1 , x_2 , and one where one is the bias value.

$$[x_1 \quad x_2 \quad 1]$$

- Each input is multiplied by weight with respect to the first and second model to obtain their probability of being in the positive region in each model.

So, we will multiply our inputs by a matrix of weight using matrix multiplication.

$$[x_1 \quad x_2 \quad 1] = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix} = [\text{score} \quad \text{score}]$$

Training FF DNN

- After that, we will take the sigmoid of our scores and gives us the probability of the point being in the positive region in both models.

$$\frac{1}{1 + e^{-x}}[\text{score score}] = \text{probability}$$

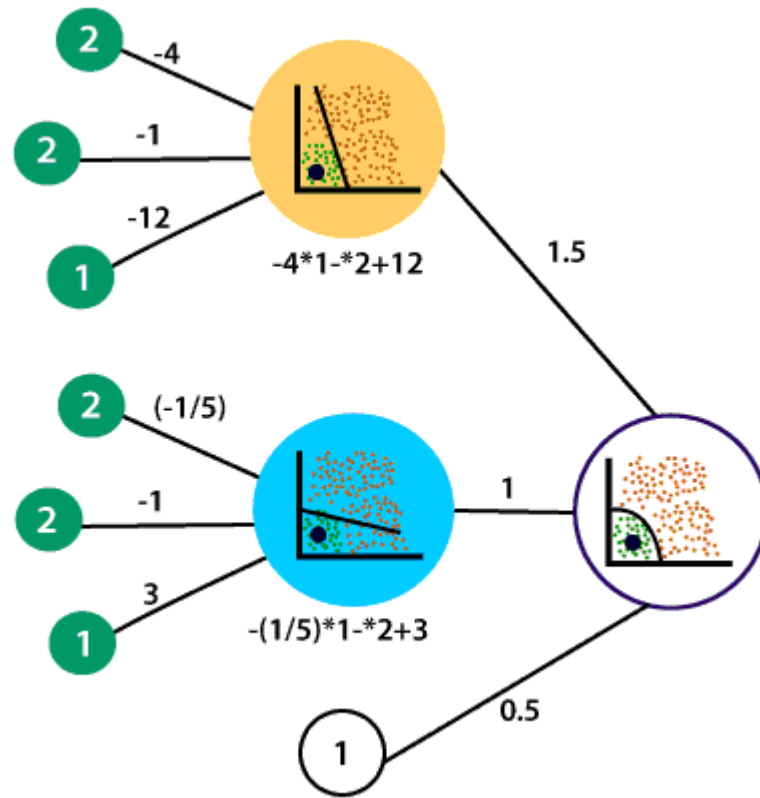
- We multiply the probability which we have obtained from the previous step with the second set of weights.

$$[\text{probability} \quad \text{probability} \quad 1] \times \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \\ w_{31} & w_{32} \end{bmatrix} = [\text{score}]$$

- And as we know to obtain the probability of the point being in the positive region of this model, we take the sigmoid and thus producing our final output in a feed-forward process.

$$\frac{1}{1 + e^{-x}}[\text{score}] = [\text{probability}]$$

Training FF DNN – Ex1



$$\begin{bmatrix} 2 & 2 & 1 \end{bmatrix} \times \begin{bmatrix} -4 & w_{12} \\ -1 & w_{22} \\ 12 & w_{32} \end{bmatrix}$$

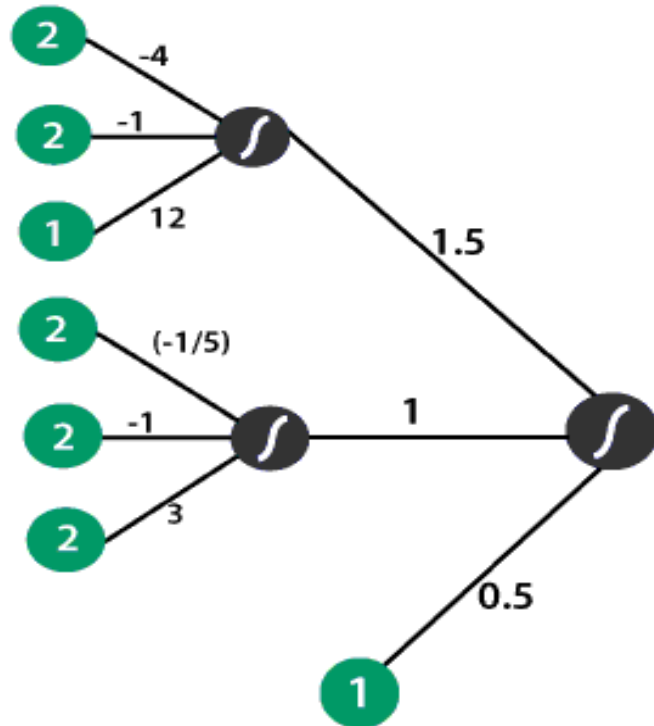
$$\begin{bmatrix} 2 & 2 & 1 \end{bmatrix} \times \begin{bmatrix} -4 & -1/5 \\ -1 & -1 \\ 12 & 3 \end{bmatrix}$$

$$\begin{bmatrix} 2(-4) + 2(-1) + 1(12) \\ 2(-4) + 2(-1) + 1(12) \end{bmatrix}$$

$$\begin{bmatrix} 2 & 0.6 \end{bmatrix}$$

$$\begin{bmatrix} \frac{1}{1+\frac{1}{e^x}} & \frac{1}{1+\frac{1}{e^x}} \end{bmatrix} = \begin{bmatrix} \frac{1}{1+\frac{1}{e^2}} & \frac{1}{1+\frac{1}{e^{0.6}}} \end{bmatrix} = [0.88 \ 0.64]$$

Training FF DNN – Ex1



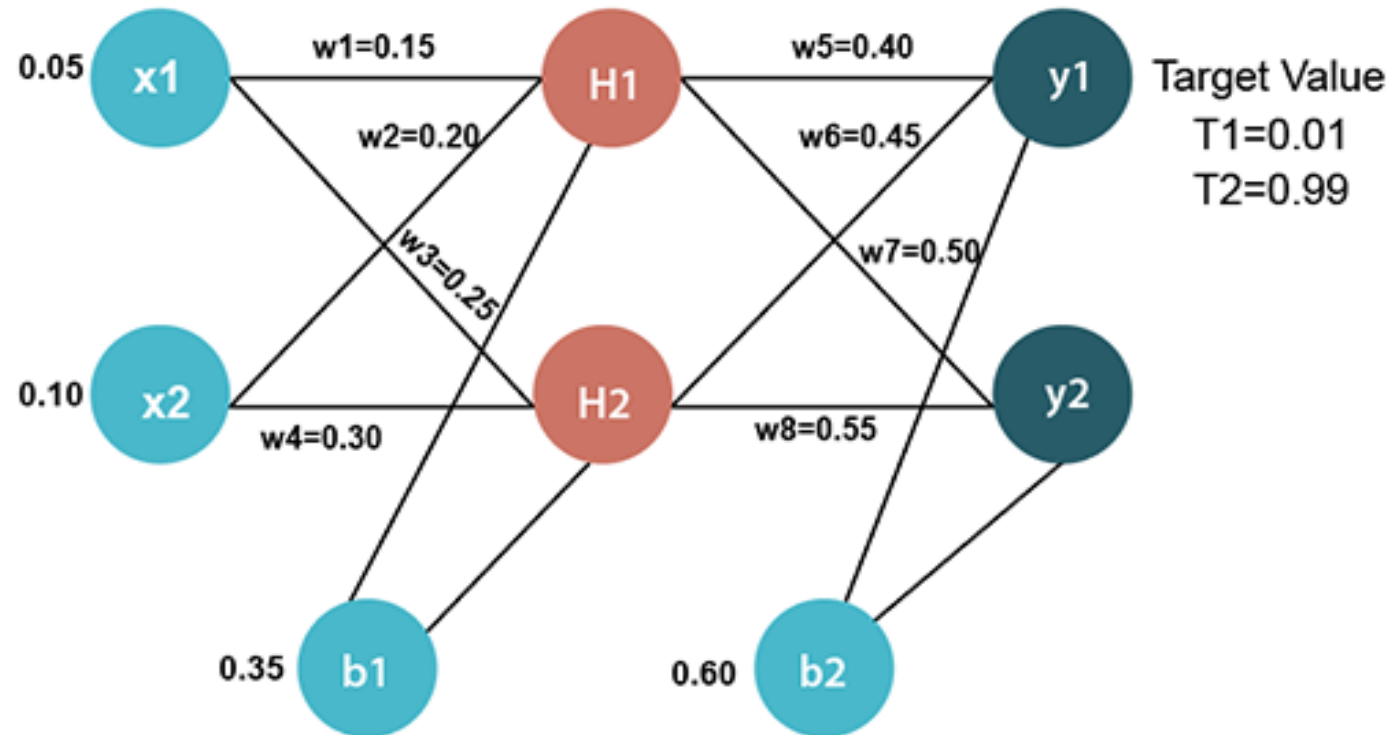
Now, we have to multiply our probabilities from the first layer with the second set of weights as

$$[0.88 \ 0.64 \ 1] \times \begin{bmatrix} 1.5 \\ 1 \\ 0.5 \end{bmatrix} = [0.88(1.5) + (0.64)(1) + 1(0.5)] = 2.46$$

Now, we will take the sigmoid of our final score

$$\frac{1}{1 + \frac{1}{e^{2.46}}} = [0.92]$$

Training FF DNN – Ex2

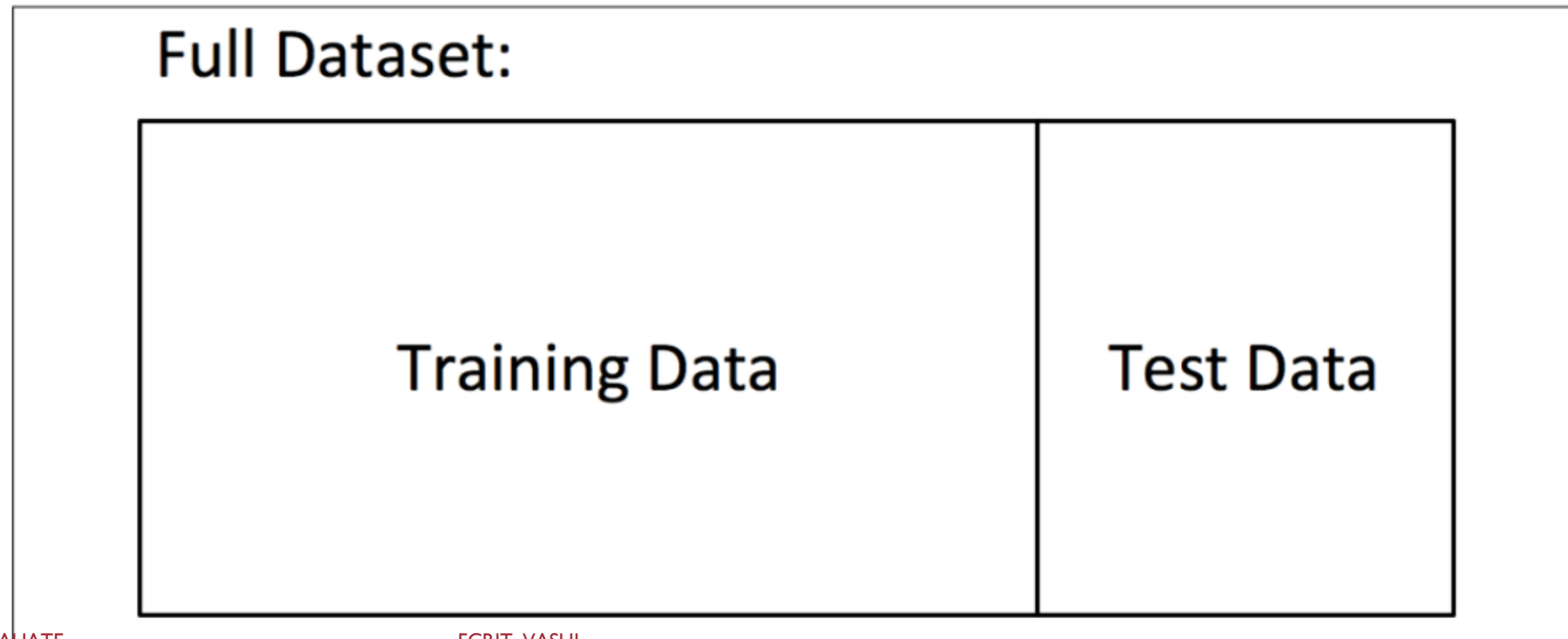




REGULARIZATION

MODEL EVALUATION

- We often split our data into nonoverlapping training and test sets in order to fairly evaluate our model

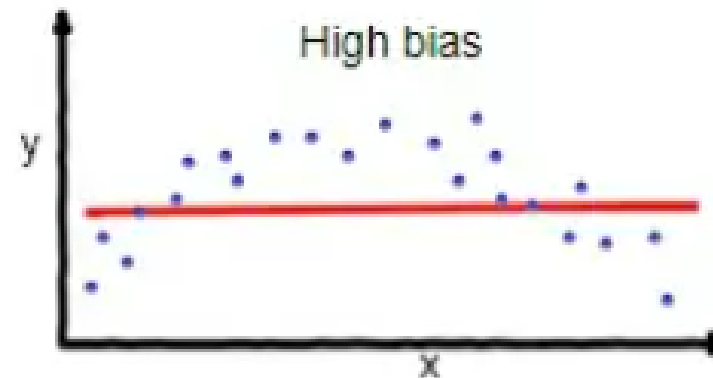


INTRODUCTION

- Whether we're a person or a computer, learning general rules about a subject from a finite set of examples is a tough challenge.
- If we don't pay enough attention to the details of the examples, our rules will be too general, and when we get to working on new data we're likely to come to wrong conclusions.
- On the other hand, if we pay too much attention to the details in the examples, our rules will be too specific, and again we'll be likely to come to wrong conclusions.
- These phenomena are respectively called underfitting and overfitting.

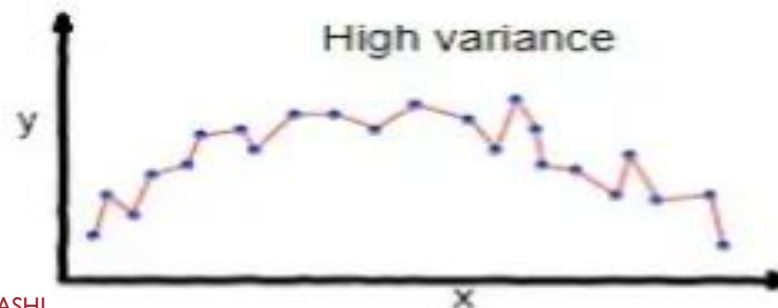
UNDERFITTING

- **Underfitting** happens when a model is unable to capture the underlying pattern of the data.
- These models usually have high bias and low variance.
- It happens when we have very less amount of data to build an accurate model or when we try to build a linear model with a nonlinear data.
- Also, these kind of models are very simple to capture the complex patterns in data like Linear and logistic regression:



OVERFITTING

- **overfitting** happens when our model captures the noise along with the underlying pattern in data.
- It happens when we train our model a lot over noisy dataset.
- These models have low bias and high variance.
- These models are very complex like Decision trees which are prone to overfitting.



WHY IS BIAS VARIANCE TRADEOFF?

- Bias is the difference between the average prediction of our model and the correct value which we are trying to predict.
- Variance is the variability of model prediction for a given data point or a value which tells us spread of our data.
- If our model is too simple and has very few parameters then it may have high bias and low variance.
- On the other hand if our model has large number of parameters then it's going to have high variance and low bias.

REGULARIZATION

Full Dataset:

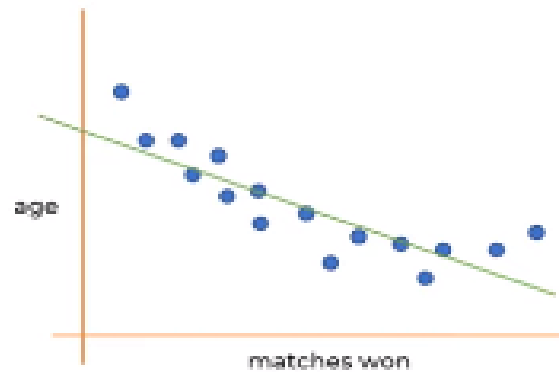
Training Data	Validation Data	Test Data
---------------	-----------------	-----------

- We always want to squeeze as much information as we can out of our training data, stopping just short of overfitting.
- The techniques for controlling overfitting are collectively known as regularization.
- It's quite likely that while we're training our data, there's a point in time where instead of learning useful features, we start overfitting to the training set.
- To avoid that, we want to be able to stop the training process as soon as we start overfitting, to prevent poor generalization.
- To do this, we divide our training process into epochs.
- **An epoch is a single iteration over the entire training set.**
- In other words, if we have a training set of size d and we are doing mini-batch gradient descent with batch size b ,
 - then an epoch would be equivalent to d / b model updates.
- **At the end of each epoch, we want to measure how well our model is generalizing.**
 - To do this, we use an additional validation set

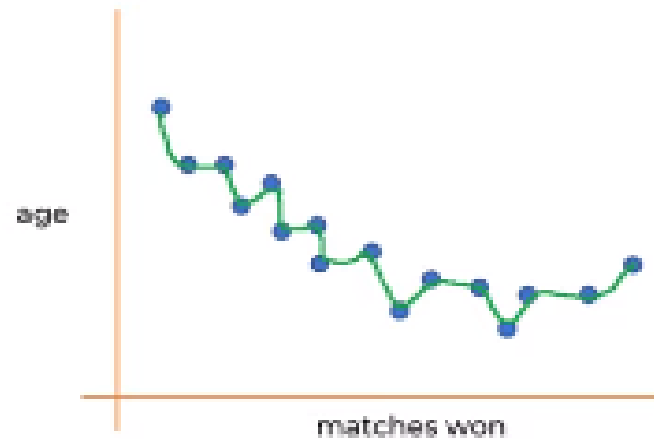
REGULARIZATION METHODS

1. L1 regularization
2. L2 regularization
3. Parameter sharing
4. Drop out
5. Weight Decay
6. Batch Normalization
7. Early Stopping
8. Data Augmentation
9. Adding Noise to input & output

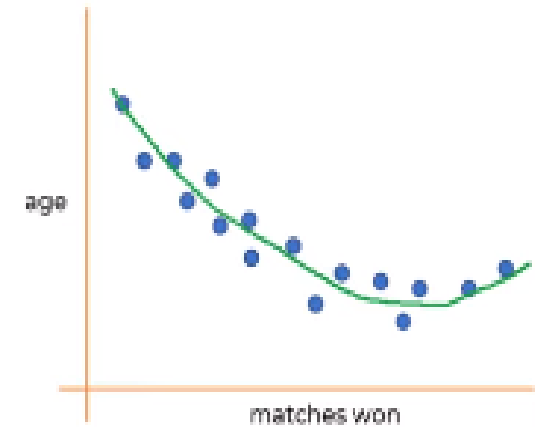
REGULARIZATION



$$\text{match won} = \theta_0 + \theta_1 * \text{age}$$



$$\text{match won} = \theta_0 + \theta_1 * \text{age} + \theta_2 * \text{age}^2 + \theta_3 * \text{age}^3 + \theta_4 * \text{age}^4$$



$$\text{match won} = \theta_0 + \theta_1 * \text{age} + \theta_2 * \text{age}^2$$

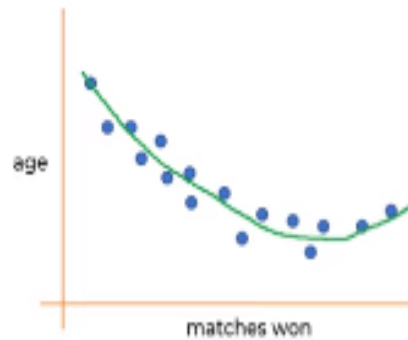
REGULARIZATION



$$\text{match won} = \theta_0 + \theta_1 * \text{age} + \theta_2 * \text{age}^2 + \theta_3 * \text{age}^3 + \theta_4 * \text{age}^4$$



Try to make θ_3 and θ_4 almost close to zero



$$\text{match won} = \theta_0 + \theta_1 * \text{age} + \theta_2 * \text{age}^2$$



Mean Squared Error

$$mse = \frac{1}{n} \sum_{i=1}^n (y_i - y_{predicted})^2$$

Mean Squared Error

$$mse = \frac{1}{n} \sum_{i=1}^n (y_i - h_{\theta}(x_i))^2$$

$$h_{\theta}(x_i) = \theta_0 + \theta_1 x_1 + \theta_2 x_2^2 + \theta_3 x_3^3$$

REGULARIZATION

$$mse = \frac{1}{n} \sum_{i=1}^n (y_i - h_{\theta}(x_i))^2 + \lambda \sum_{i=1}^n \theta_i^2$$

$$h_{\theta}(x_i) = \theta_0 + \theta_1 x_1 + \theta_2 x_2^2 + \theta_3 x_3^3$$

PARAMETER NORM PENALTIES

- Many regularization approaches are based on limiting the capacity of models,
- by adding a parameter norm penalty $\Omega(\theta)$ to the objective function J
- We denote the regularized objective function by

Regularized loss function:

$$\tilde{J}(\theta; X, y) = \underbrace{J(\theta; X, y)}_{\text{Unregularized loss function}} + \underbrace{\alpha \Omega(\theta)}_{\text{Regularizer on weights and bias vectors}}$$

Training data

Convention:
 w denotes only the weights used in regularization
 θ denotes all the parameters

Typical:
bias vectors are not used for regularization

REGULARIZATION

L2 Regularization

$$mse = \frac{1}{n} \sum_{i=1}^n (y_i - h_{\theta}(x_i))^2 + \lambda \sum_{i=1}^n \theta_i^2$$

$$h_{\theta}(x_i) = \theta_0 + \theta_1 x_1 + \theta_2 x_2^2 + \theta_3 x_3^3$$

L1 Regularization

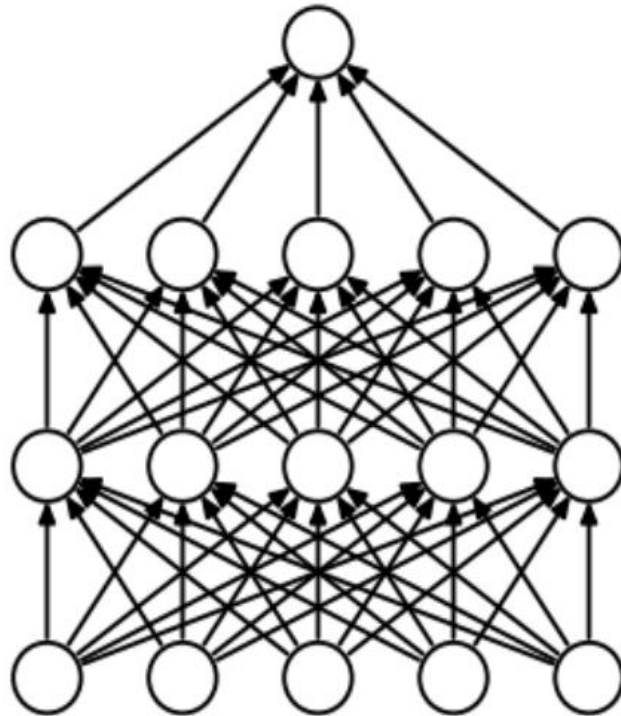
$$mse = \frac{1}{n} \sum_{i=1}^n (y_i - h_{\theta}(x_i))^2 + \lambda \sum_{i=1}^n |\theta_i|$$

$$h_{\theta}(x_i) = \theta_0 + \theta_1 x_1 + \theta_2 x_2^2 + \theta_3 x_3^3$$

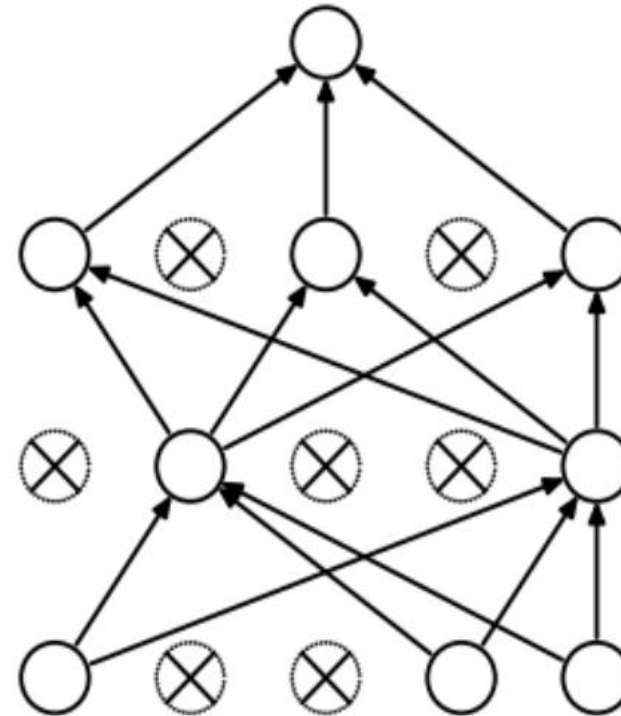
DROP OUT

- During training, some layer outputs are ignored or dropped at random.
- This makes the layer appear and is regarded as having a different number of nodes and connectedness to the preceding layer.
- In practice, each layer update during training is carried out with a different perspective of the specified layer.
- Dropout makes the training process noisy, requiring nodes within a layer to take on more or less responsible for the inputs on a probabilistic basis.
- According to this conception, dropout may break apart circumstances in which network tiers co-adapt to fix mistakes committed by prior layers, making the model more robust.

DROP OUT



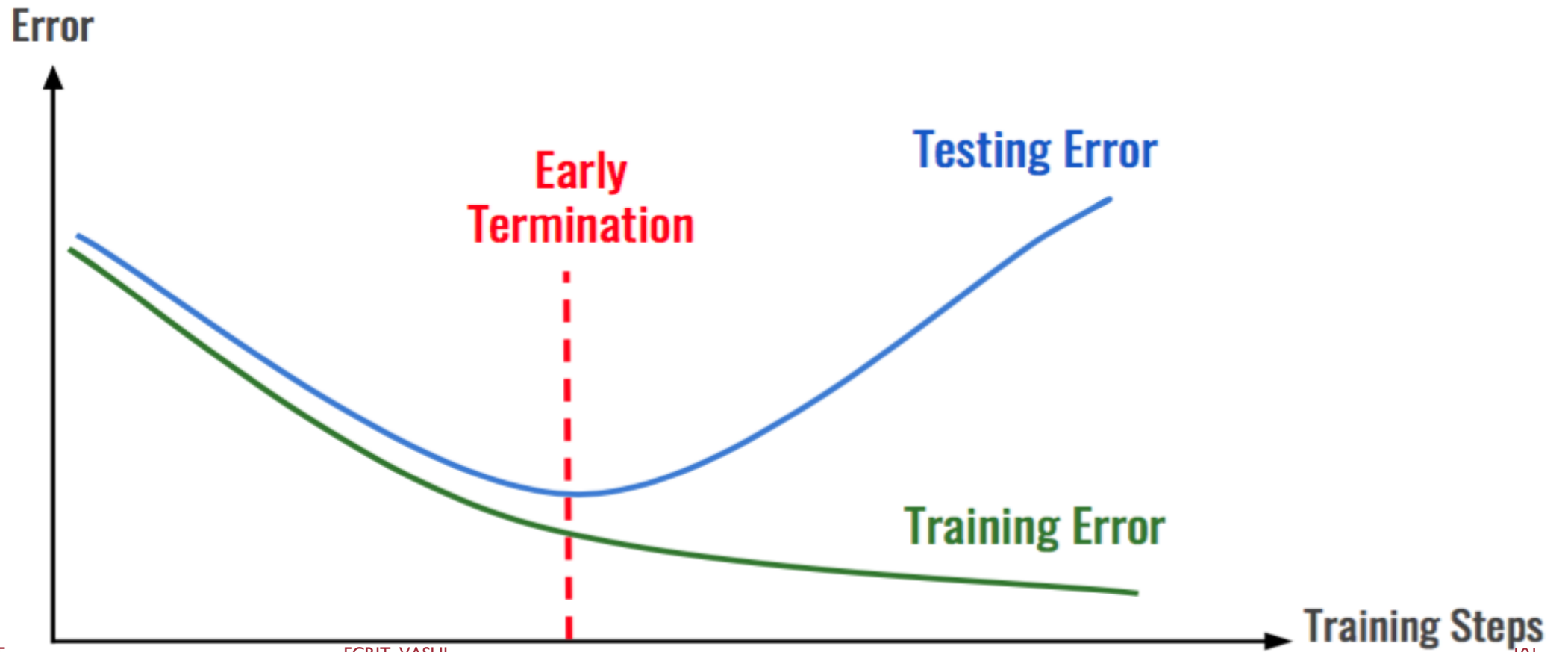
(a) Standard Neural Net



(b) After applying dropout.

- ❑ It can't rely on one input as it might be randomly dropped out.
- ❑ Neurons will not learn redundant details of inputs

EARLY STOPPING



PARAMETER SHARING

- Instead of penalizing model parameters, it forces a group of parameters to be equal
- This can be seen as a way to apply previous domain knowledge to the training process
- **CNN takes advantage** of the spatial structure of images by sharing parameters across different locations in the input

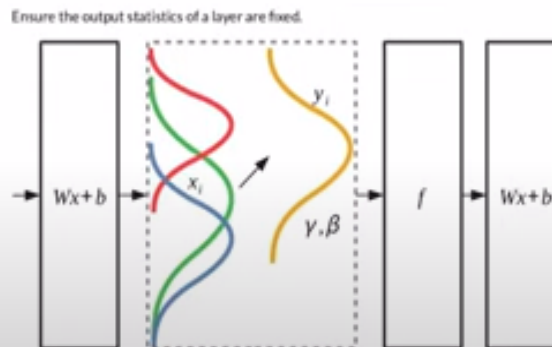
BATCH NORMALIZATION

- It fixes means and variances of the input by bringing the feature in the same range

Problem: The distribution of each layer's input changes during training

Solution: Fix the distribution

Batch normalization is a key solution to avoid/minimize the vanishing gradient problem

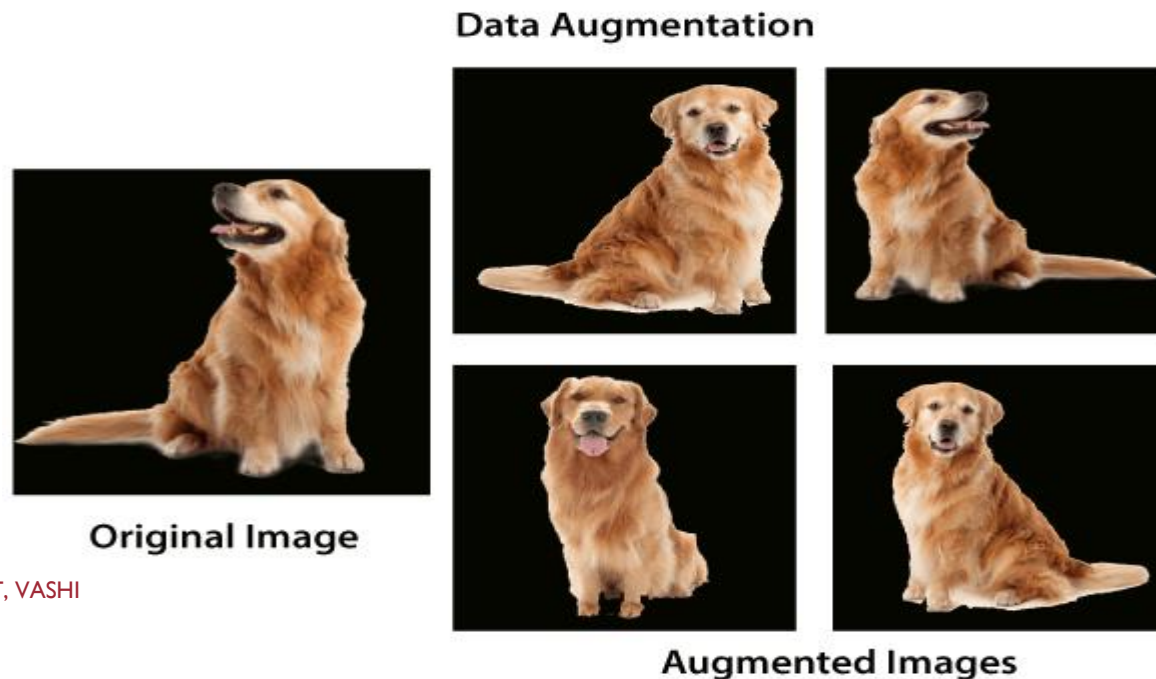


CHALLENGE OF SMALL TRAINING DATASETS

- The first problem is that the network may effectively memorize the training dataset.
- The second problem is that a small dataset provides less opportunity to describe the structure of the input space and its relationship to the output

DATA AUGMENTATION

- It refers to the process of generating new training examples to our dataset.
- More training data means lower model variance
- It can also be seen as a form of noise injection in the training dataset



HOW AND WHERE TO ADD NOISE

- The most common type of noise used during training is the addition of **Gaussian noise** to input variables.
- Gaussian noise, or white noise, has a mean of zero and a standard deviation of one and can be generated as needed using a pseudorandom number generator.
- The amount of noise added (eg. the spread or standard deviation) is a configurable hyperparameter.
- Too little noise has no effect, whereas too much noise makes the mapping function too challenging to learn.
- Although additional noise to the inputs is the most common and widely studied approach, random noise can be added to other parts of the network during training. Some examples include:
 - **Add noise to activations**, i.e. the outputs of each layer.
 - **Add noise to weights**, i.e. an alternative to the inputs.
 - **Add noise to the gradients**, i.e. the direction to update weights.
 - **Add noise to the outputs**, i.e. the labels or target variables.